

Apuntes – Clase 7

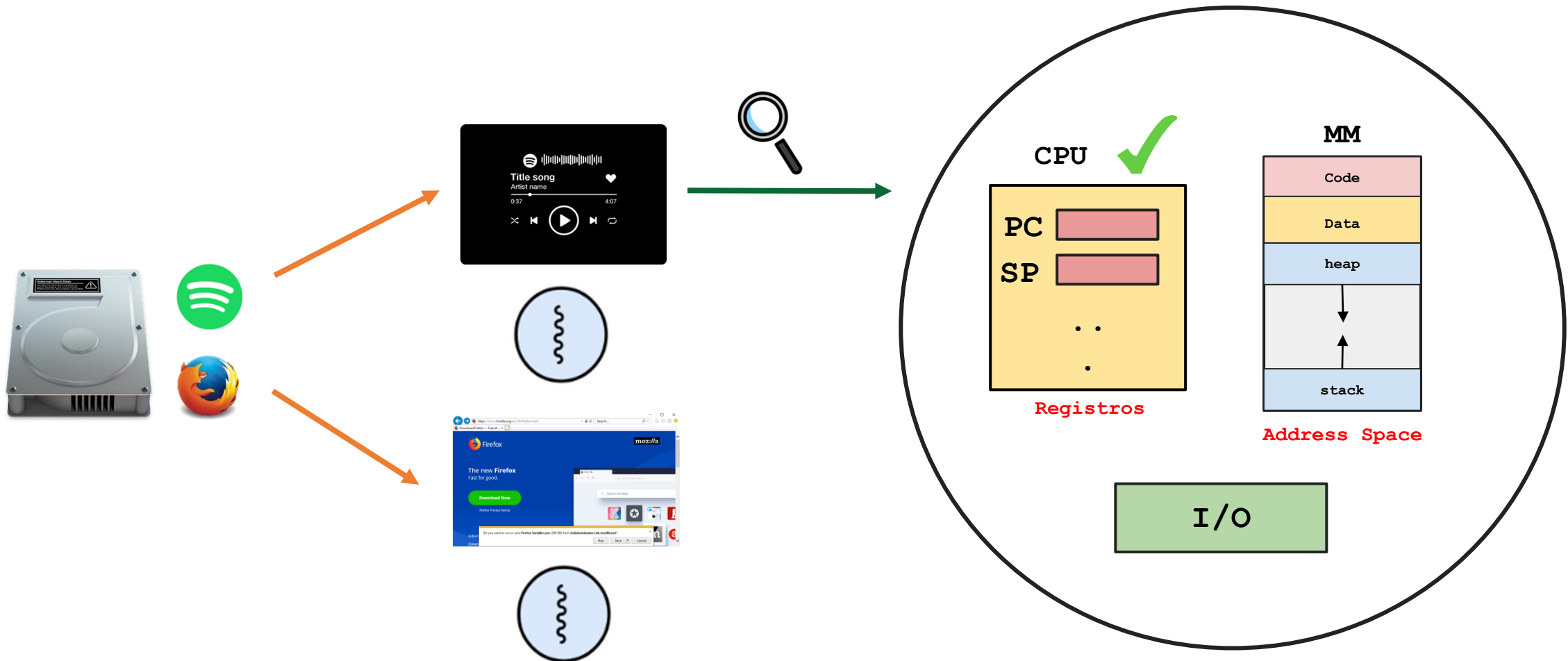
Memoria virtual

12/03/2026

Proceso

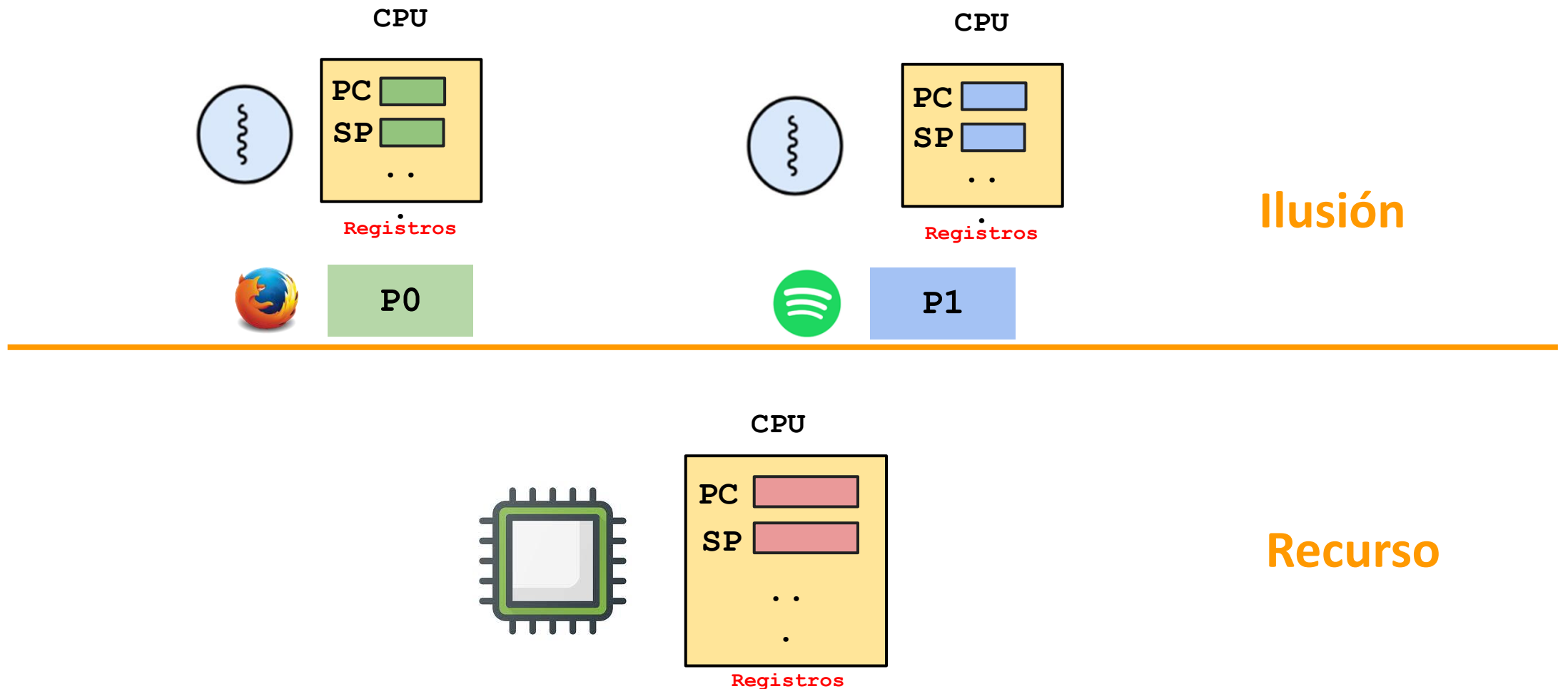
Un proceso es un **programa en ejecución**.

Proceso = CPU + Memory + I/O Info



Virtualización de CPU

Cada proceso cree que **tiene su propia CPU**.



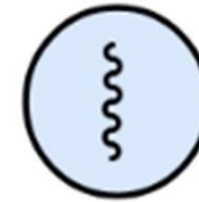
Implementación de la Virtualización

¿Cómo se implementa la virtualización?

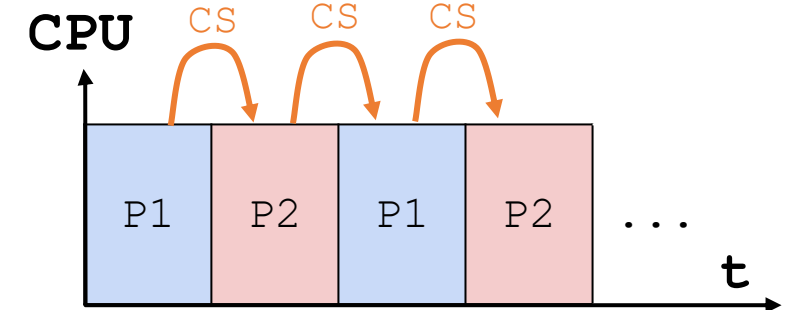
Process
management



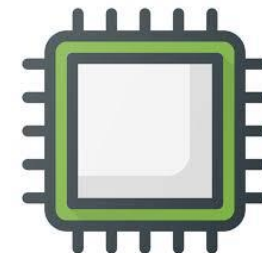
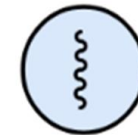
Abstracción



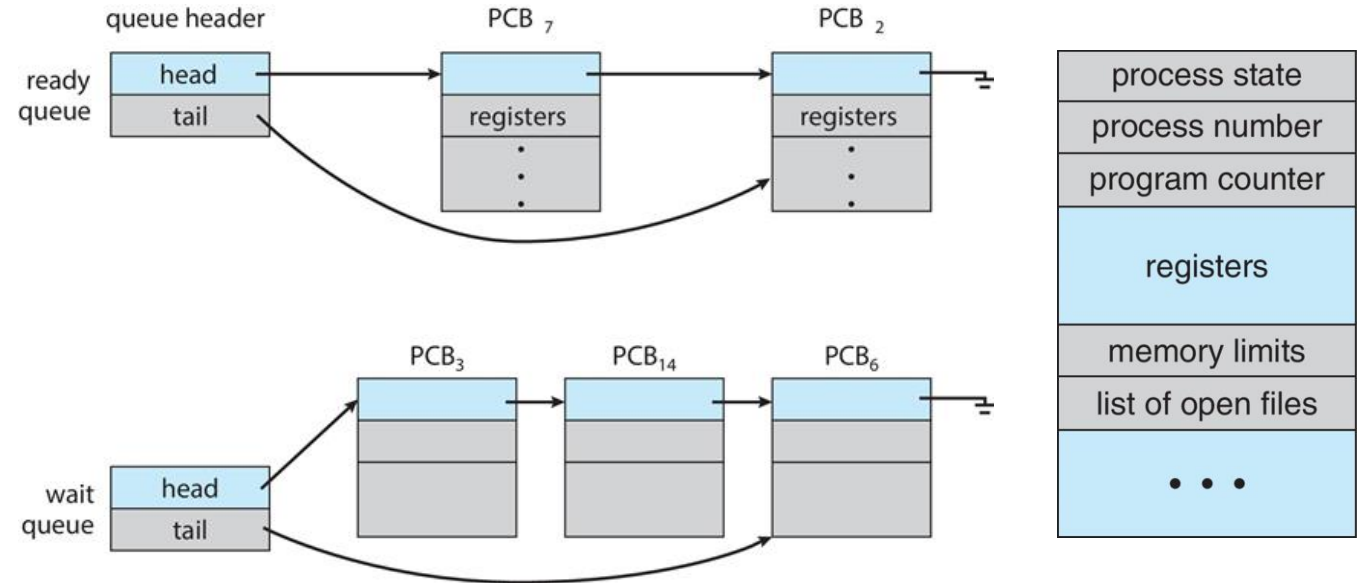
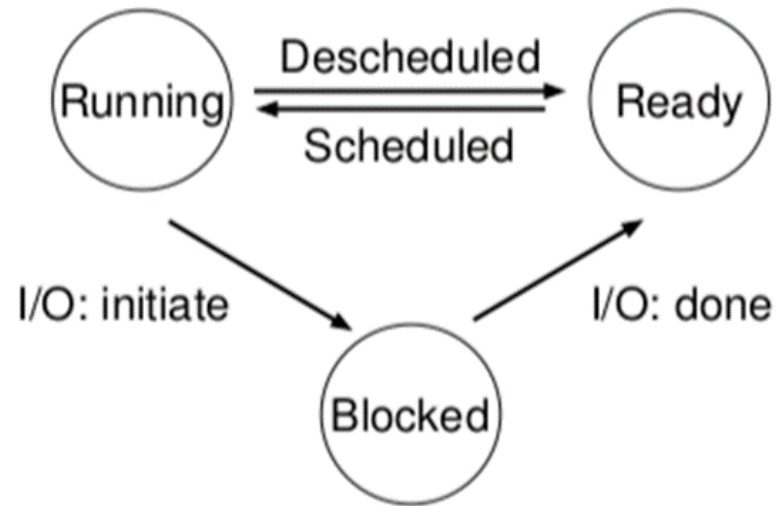
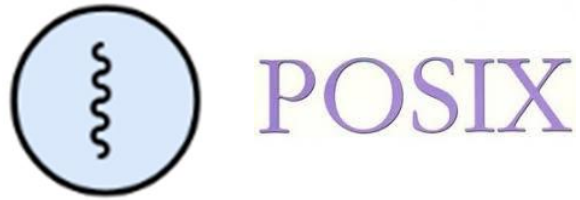
Políticas



Mecanismos



Abstracción CPU - Proceso

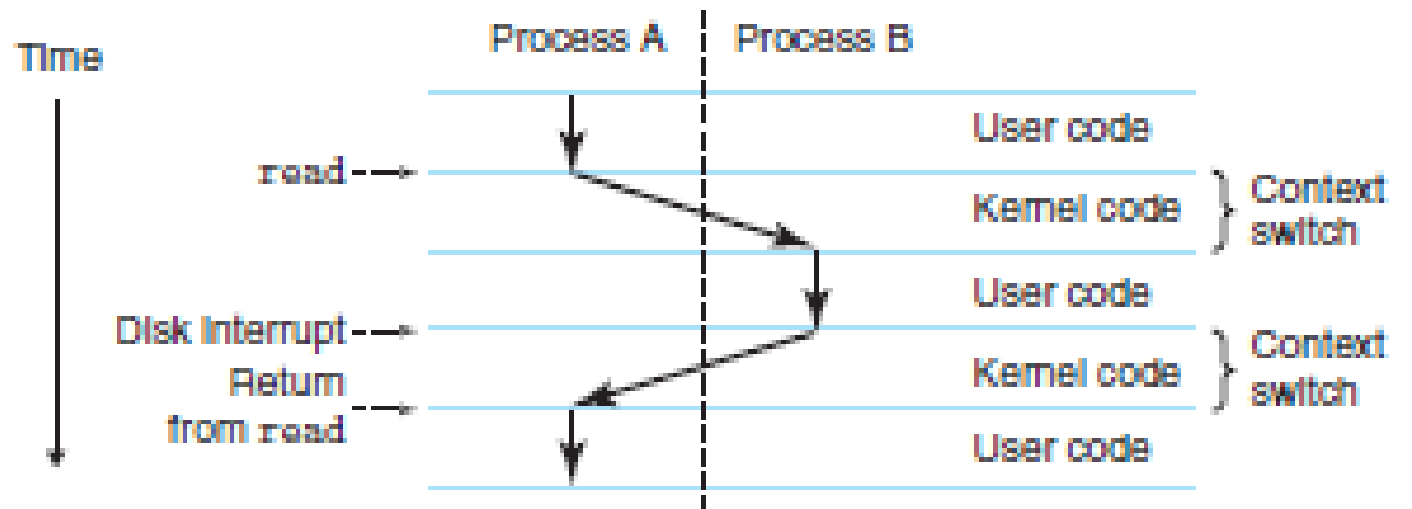
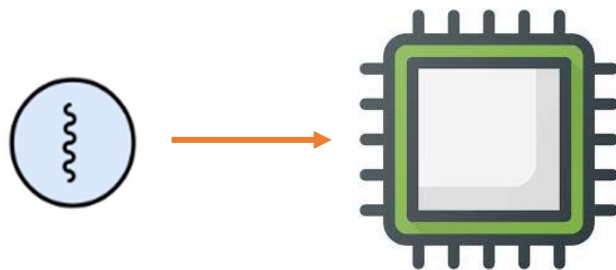
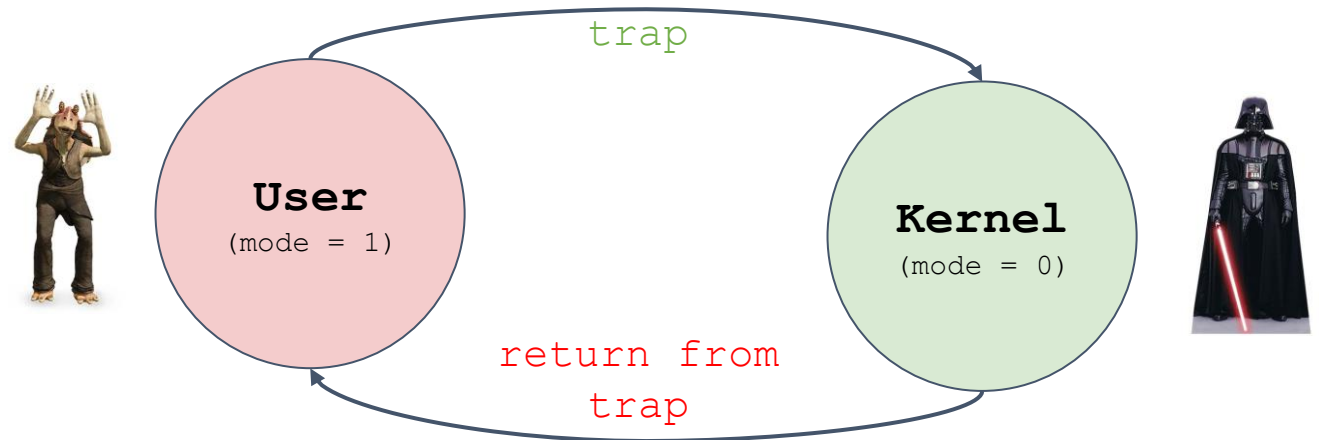


Mecanismos CPU - Proceso

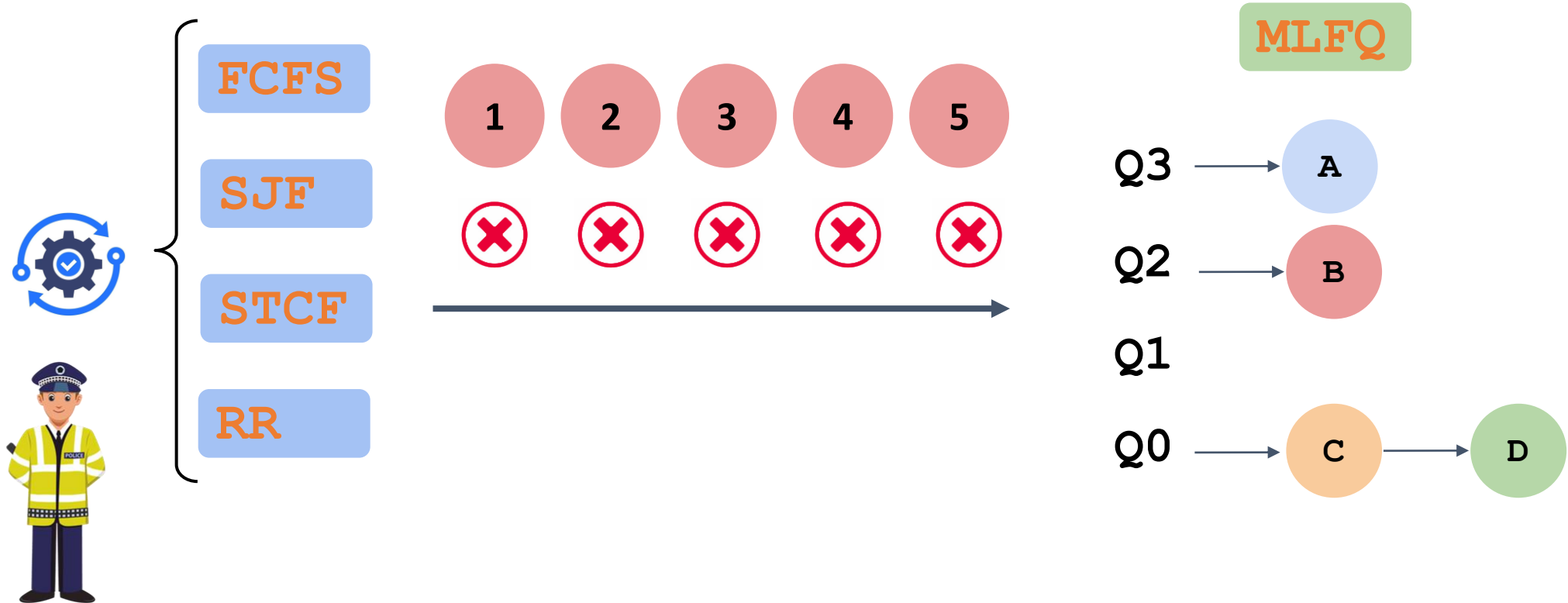
Limited

Direct

Execution

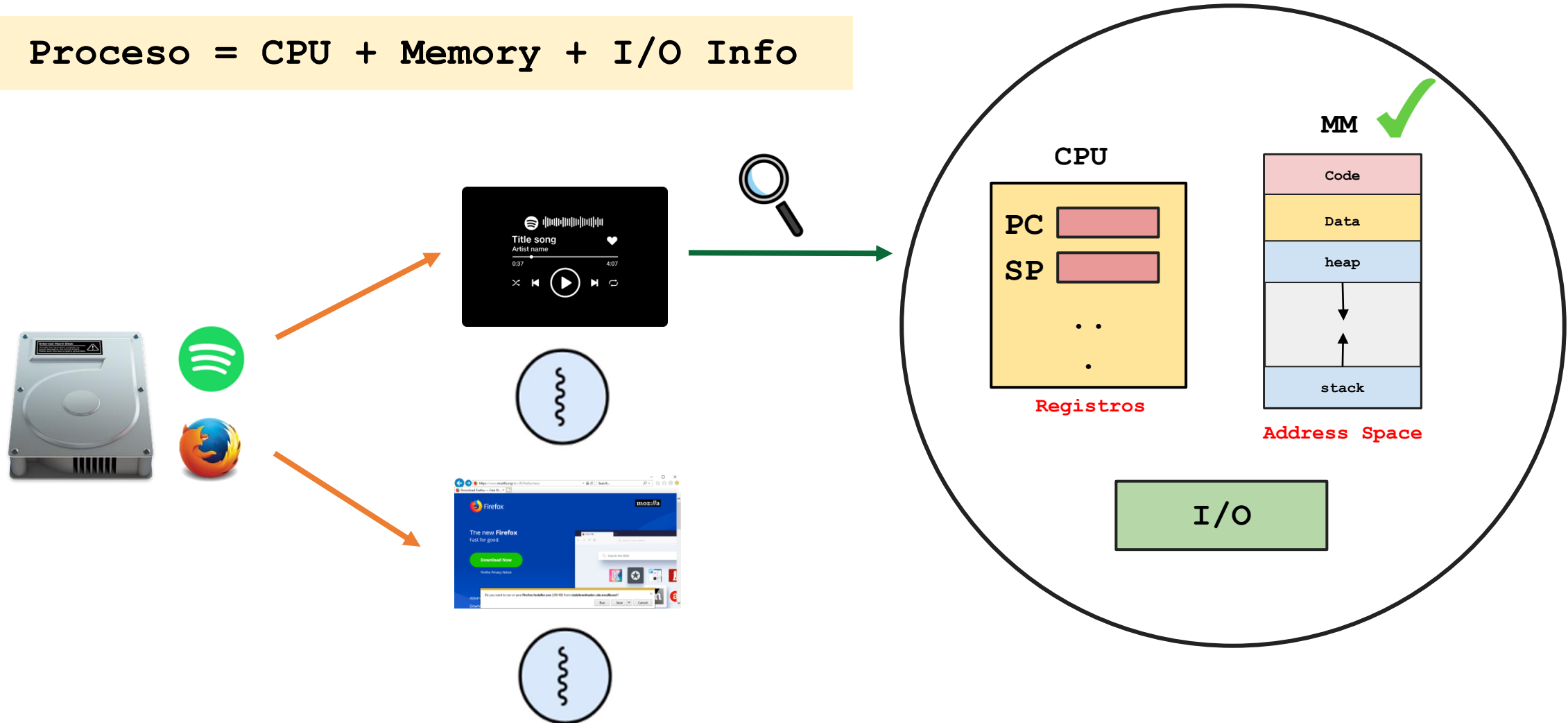


Políticas CPU - Proceso



Virtualización de Memoria - Contextualización

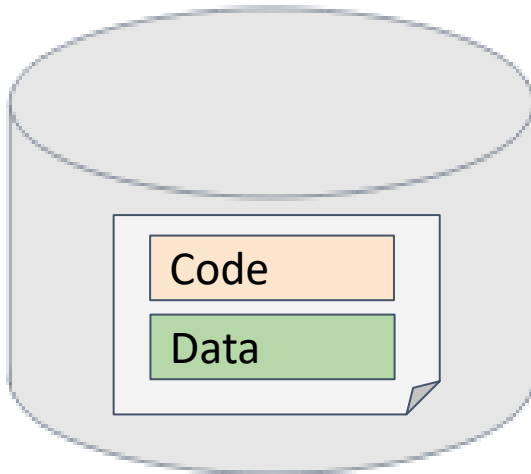
Proceso = CPU + Memory + I/O Info



Virtualización de Memoria - Contextualización



mem.c



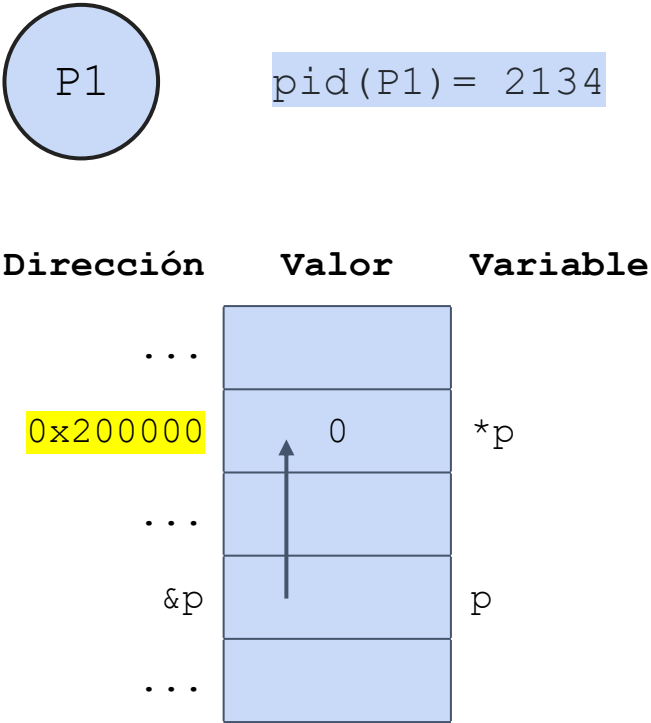
```
#include <unistd.h>
#include <stdio.h>
#include <stdlib.h>
#include "common.h"

int main(int argc, char *argv[]) {
    if (argc != 2) {
        fprintf(stderr, "usage: mem <value>\n");
        exit(1);
    }
    int *p;
    p = malloc(sizeof(int));
    assert(p != NULL);
    printf("(%d) addr pointed to by p: %p\n", (int) getpid(), p);
    *p = atoi(argv[1]); // assign value to addr stored in p
    while (1) {
        Spin(1);
        *p = *p + 1;
        printf("(%d) value of p: %d\n", getpid(), *p);
    }
    return 0;
}
```

Virtualización de Memoria - Contextualización

Pregunta: ¿Cómo funciona el programa?

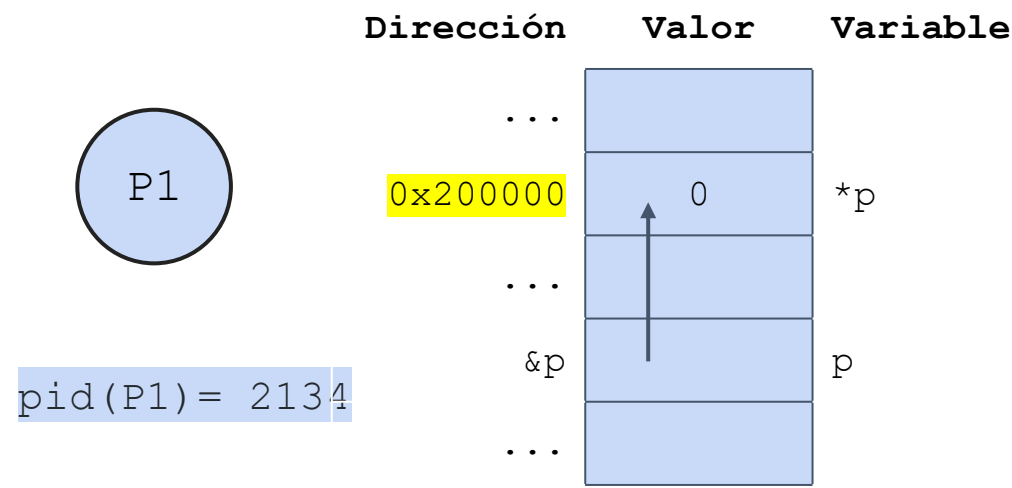
```
prompt> ./mem 1
(2134) address pointed to by p: 0x200000
(2134) p: 1
(2134) p: 2
(2134) p: 3
(2134) p: 4
(2134) p: 5
^C
```



Virtualización de Memoria - Contextualización

Pregunta: ¿Cómo funciona el programa?

```
prompt> ./mem 1
(2134) address pointed to by p: 0x200000
(2134) p: 1
(2134) p: 2
(2134) p: 3
(2134) p: 4
(2134) p: 5
^C
```

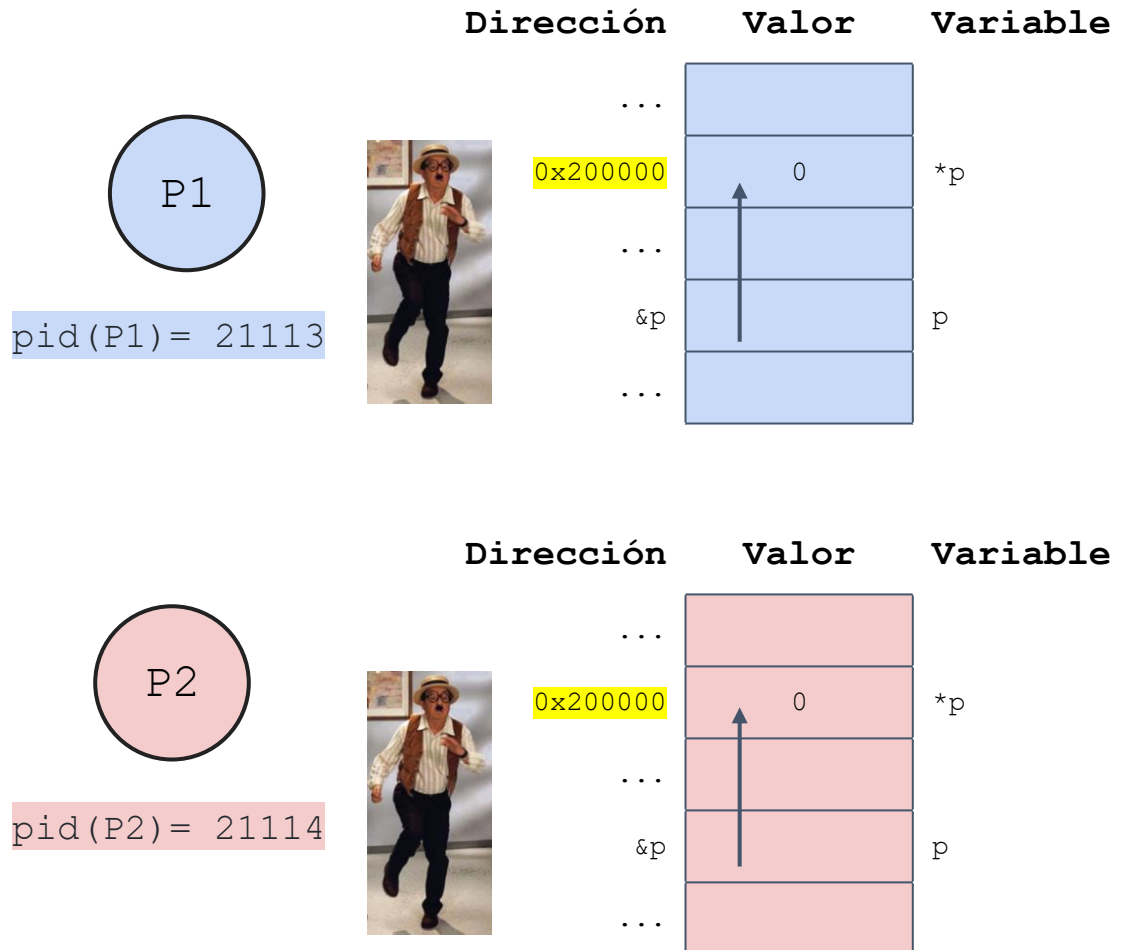


Virtualización de Memoria - Contextualización

Pregunta: ¿Que pasa si dos usuarios corren simultáneamente el programa?

```
prompt> ./mem 1 & ./mem 1 &  
[1] 24113  
[2] 24114  
(24113) address pointed to by p: 0x200000  
(24114) address pointed to by p: 0x200000  
(24113) p: 1  
(24114) p: 1  
(24114) p: 2  
(24113) p: 2  
(24113) p: 3  
(24114) p: 3  
(24113) p: 4  
(24114) p: 4  
...
```

¿Por que son iguales las direcciones si los procesos son diferentes?

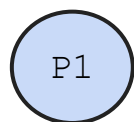


Virtualización de Memoria - Contextualización

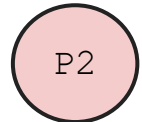
Pregunta: ¿Que pasa si dos usuarios corren simultáneamente el programa?

```
prompt> ./mem 1 & ./mem 1 &  
[1] 24113  
[2] 24114  
(24113) address pointed to by p: 0x200000  
(24114) address pointed to by p: 0x200000  
(24113) p: 1  
(24114) p: 1  
(24114) p: 2  
(24113) p: 2  
...
```

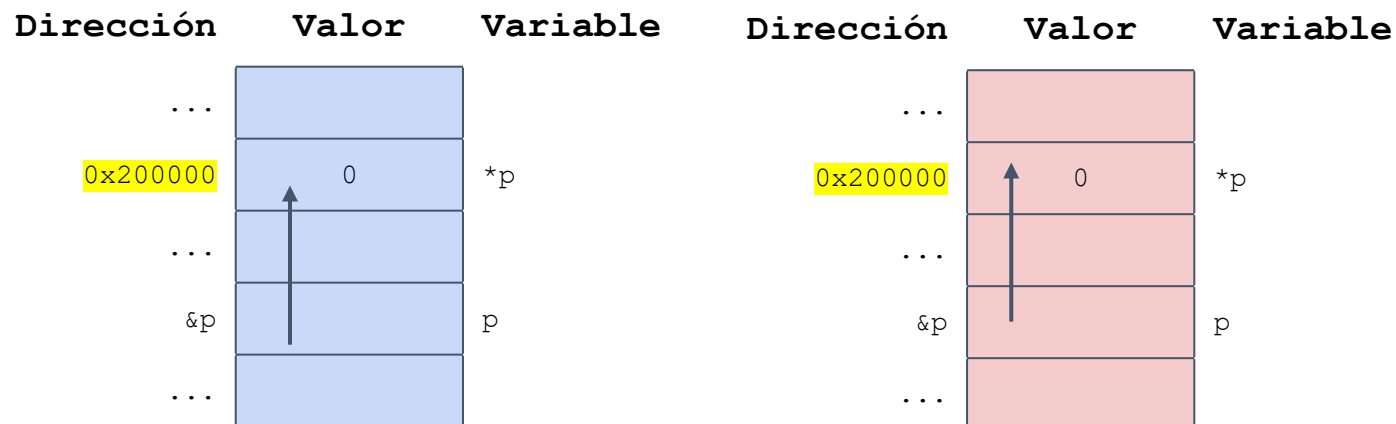
¿Por que son iguales las direcciones si los procesos son diferentes?



pid(P1)= 21113

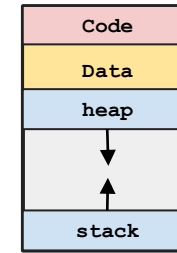
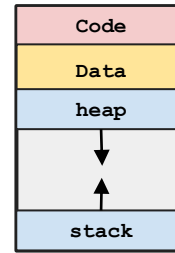
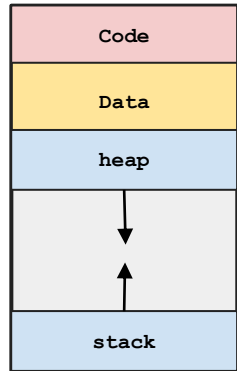


pid(P2)= 21114



Virtualización de Memoria - Contextualización

Virtualización de memoria: Cada proceso cree que **tiene su propia memoria**.



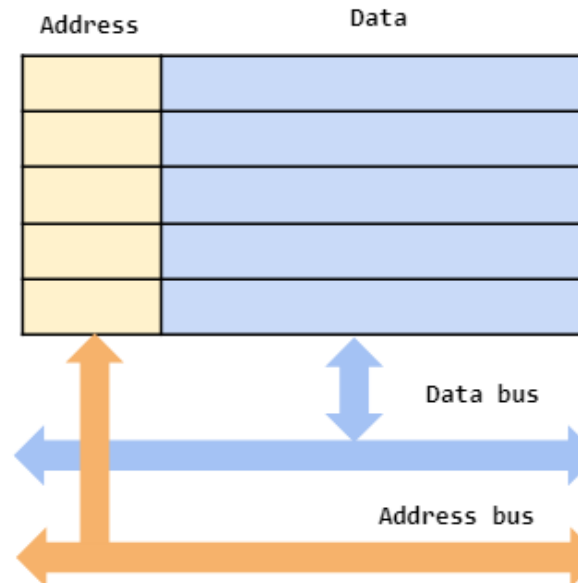
Ilusión



P0



P1

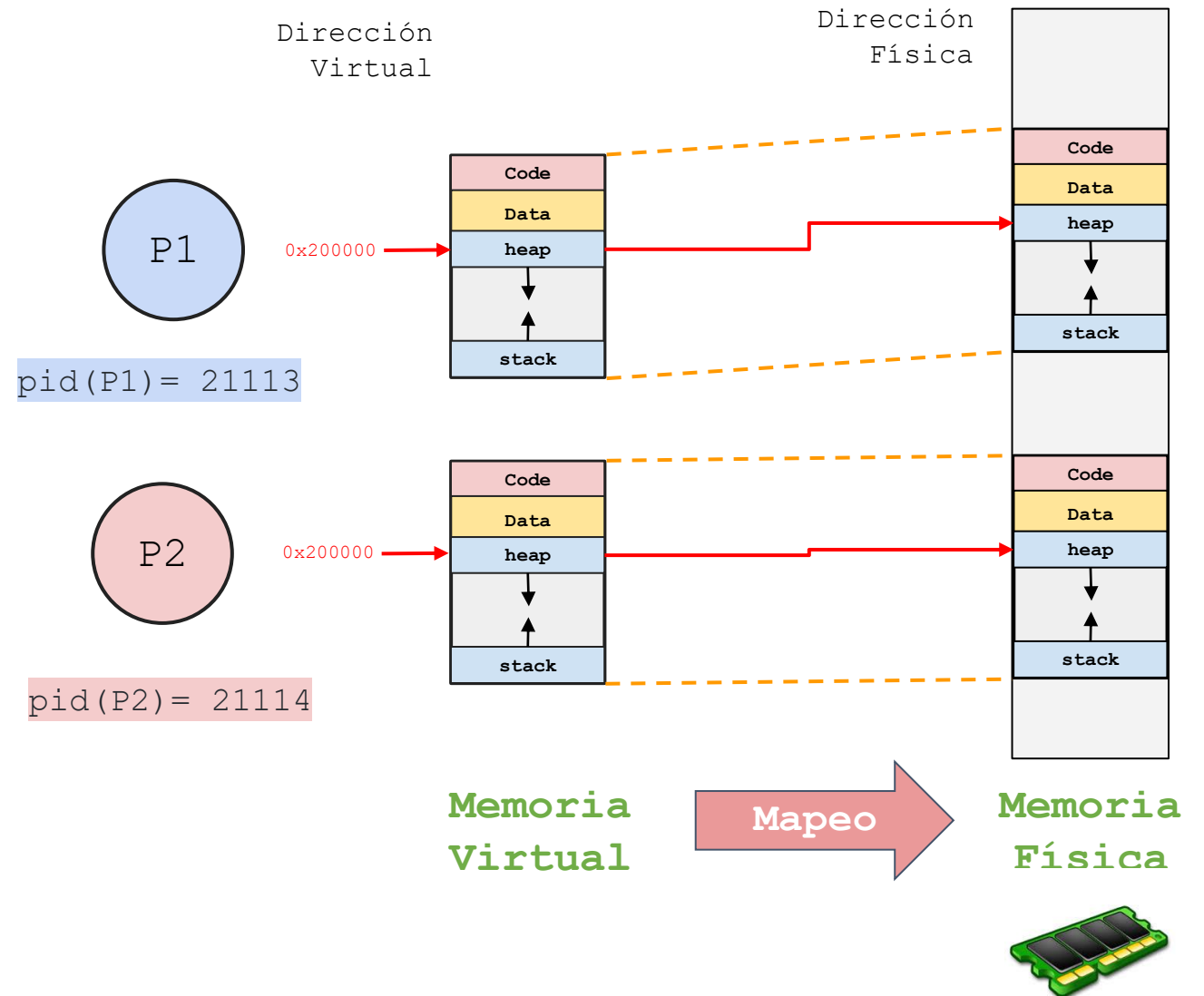


Recurso

Virtualización de Memoria - Contextualización

Virtualización de memoria:
Cada proceso cree que **tiene su propia memoria**.

```
prompt> ./mem 1 & ./mem 1 &  
[1] 24113  
[2] 24114  
(24113) address pointed to by p: 0x200000  
(24114) address pointed to by p: 0x200000  
(24113) p: 1  
(24114) p: 1  
(24114) p: 2  
(24113) p: 2  
(24113) p: 3  
(24114) p: 3  
(24113) p: 4  
(24114) p: 4  
...
```



Virtualización de Memoria - Contextualización

Hardware Address Translation - Address Translation (AT)

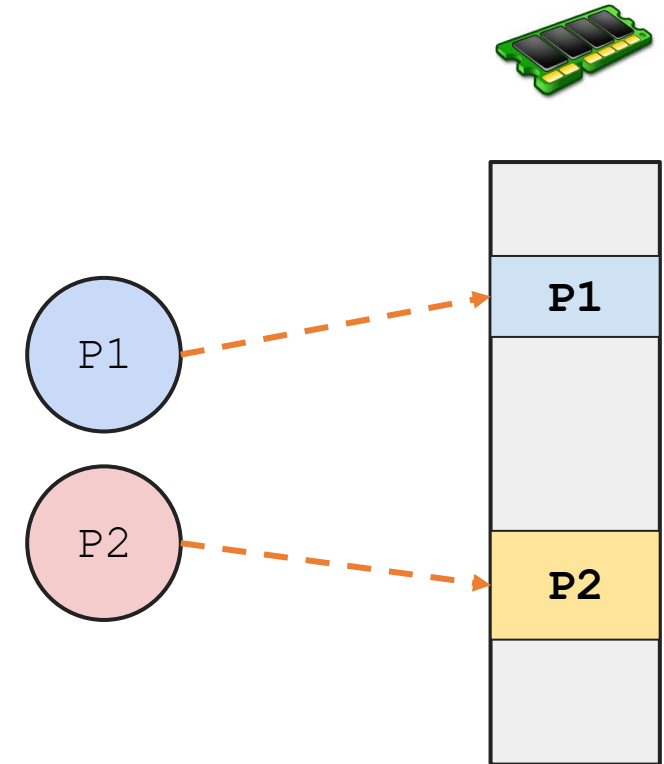
- Mecanismo mediante el cual el hardware traduce cada acceso a memoria (Instruction fetch - Load - Store) cambiando la **dirección virtual** proporcionada por la instrucción a una **dirección física** donde la información deseada se encuentra actualmente localizada.
- **Objetivo de la AT:** Dar la ilusión a un programa de que tiene su propio espacio de direcciones privado (propio).



Virtualización de Memoria - Contextualización

El sistema operativo (SO):

- **Virtualiza** la memoria física.
- **Proporciona una ilusión** para el espacio de memoria de cada proceso.
- Cada proceso **crea** que utiliza todos los recursos de memoria de la máquina.



Beneficios de la virtualización de Memoria

- **Facilidad de uso** para el programador.
- **Eficiencia** de memoria en términos de **tiempo y espacio**.
- **Protección (seguridad)**
 - Garantía de aislamiento entre procesos y SO.
 - Protección ante accesos ilegales de otros procesos

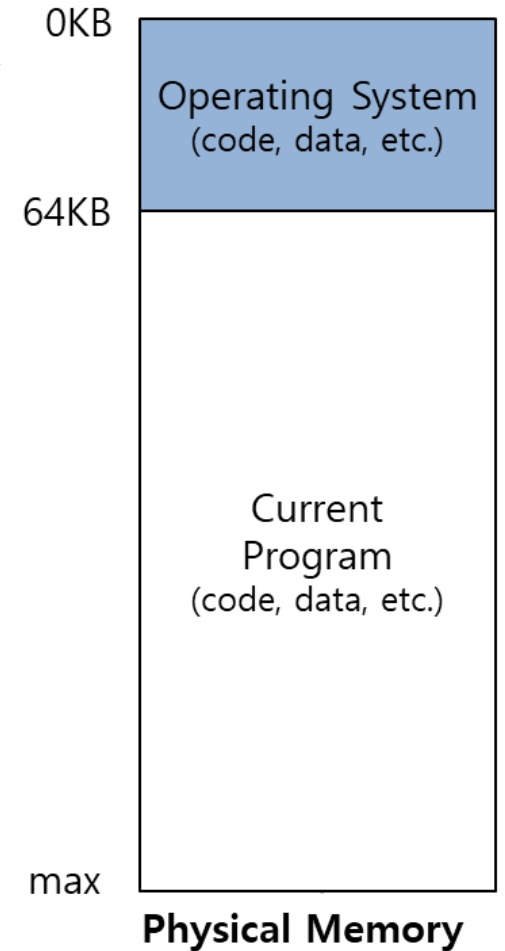


Sistemas operativos y memoria

Primeros sistemas operativos

- El sistema operativo consistía en un conjunto de rutinas (biblioteca) que se encontraba al inicio de la memoria física.
- Solo se cargaba **un solo proceso** en el resto de la memoria.

**Bajo nivel de
utilización - poco
eficiente**

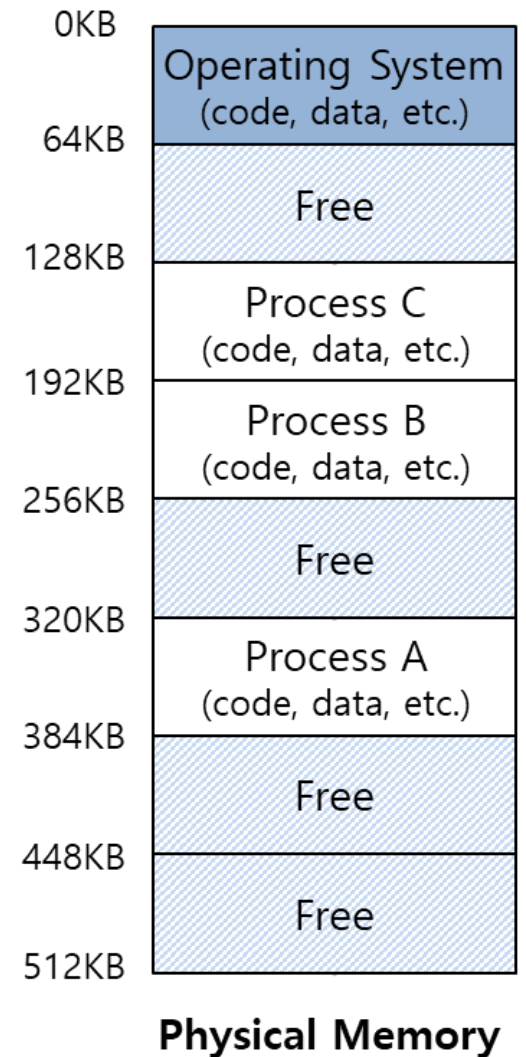


Sistemas operativos y memoria

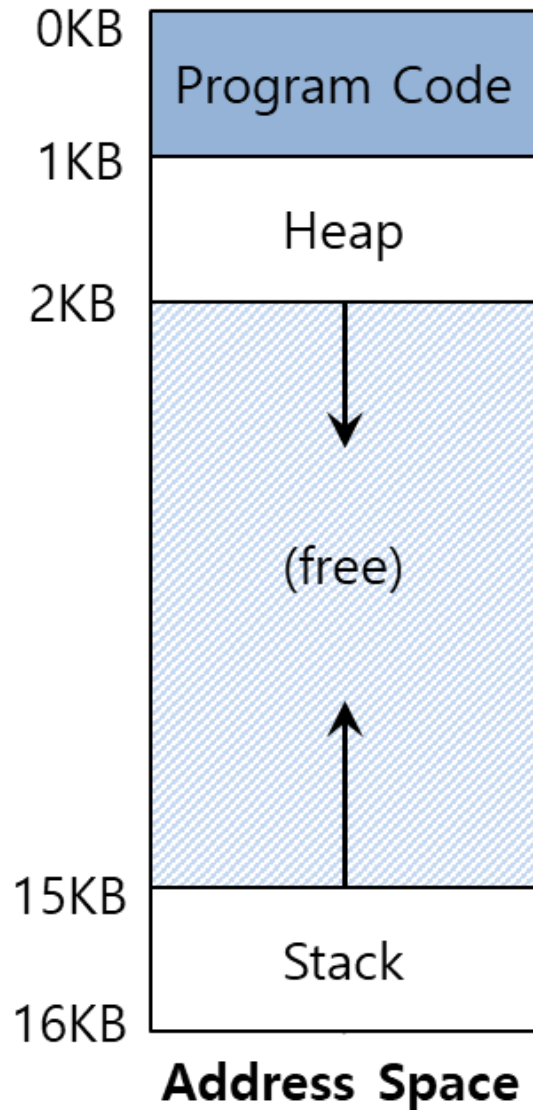
Multiprogramación - tiempo compartido

- Carga múltiples procesos en memoria
 - Ejecuta un proceso por un periodo corto de tiempo.
 - Cambia uso de CPU entre procesos.
 - Incrementa utilización y eficiencia.
- Crea un problema de **protección**.

**¡Accesos de
memoria ilegales
de otros
procesos!**

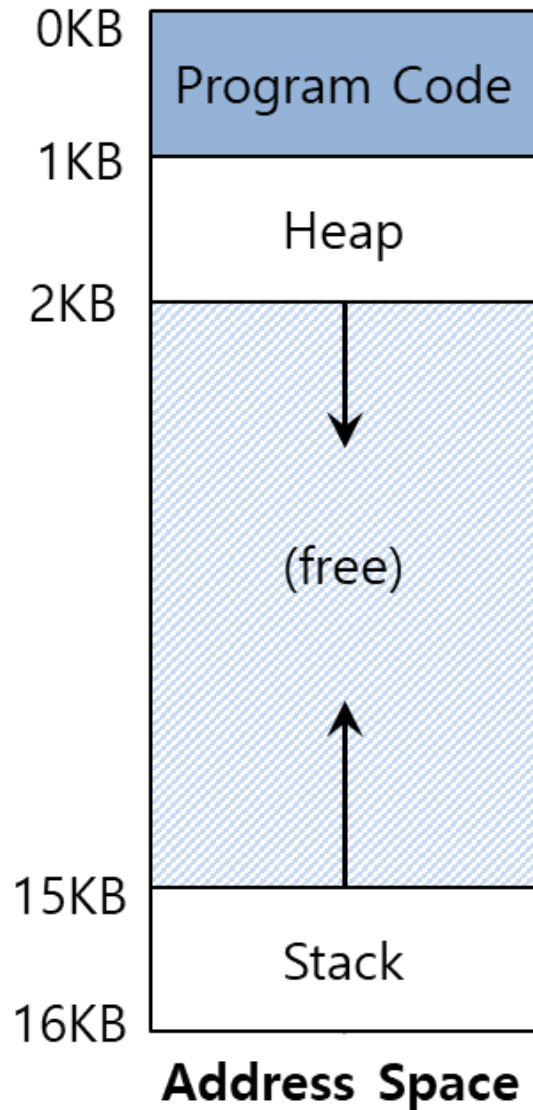


Espacio de direcciones: Address Space



- **Abstracción** de la memoria física creada por el sistema operativo
- Contiene todos los datos e instrucciones de un proceso en ejecución
 - Program code (text)
 - Data
 - Heap
 - Stack

Espacio de direcciones: Address Space



Segmentos del espacio de direcciones

Program Code	● Instrucciones ● Variables globales
Heap	● Memoria asignada dinámicamente ○ <code>malloc</code> en C ○ <code>new</code> en Java o C++
Stack	● Memoria automática ● Direcciones y valores de retorno ● Variables locales y argumentos pasados a rutinas

Espacio de direcciones: Address Space

- Todas las direcciones creadas por un programa son virtuales!
- El SO traduce las direcciones **virtuales** a direcciones **físicas**.

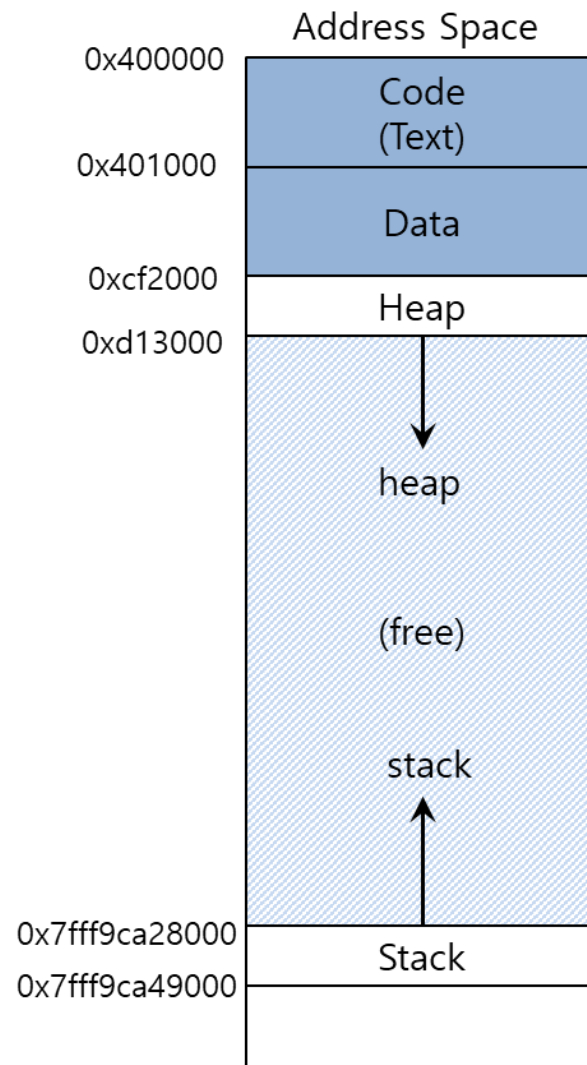
```
#include <stdio.h>
#include <stdlib.h>

int main(int argc, char *argv[]) {
    printf("location of code   : %p\n", (void *) main);
    printf("location of heap   : %p\n", (void *) malloc(1));
    int x = 3;
    printf("location of stack  : %p\n", (void *) &x);
    return x;
}
```

Ejemplo. [va.c](#) ([simulación](#))

```
location of code   : 0x40057d
location of heap   : 0xcf2010
location of stack  : 0x7fff9ca45fcc
```

The output in 64-bit Linux machine



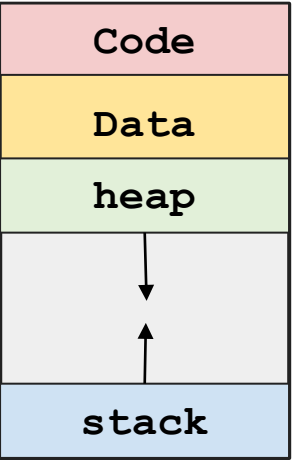
Espacio de direcciones: Address Space

Ejemplo

Dado el siguiente fragmento de código ([link](#)).

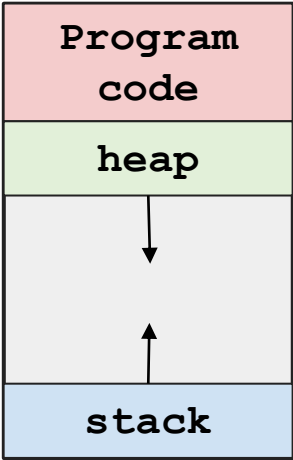
```
int x;  
int main(int argc, char *argv[]) {  
    int y;  
    int *z = malloc(sizeof(int));  
}
```

¿A qué segmento pertenece cada una de las variables mencionadas?



Variable	Location (heap, stack, code)
x	
main	
y	
z	

Ostep



Hardware Address Translation - Address Translation (AT)

Mecanismo mediante la traducción de **direcciones virtuales** a **físicas**.



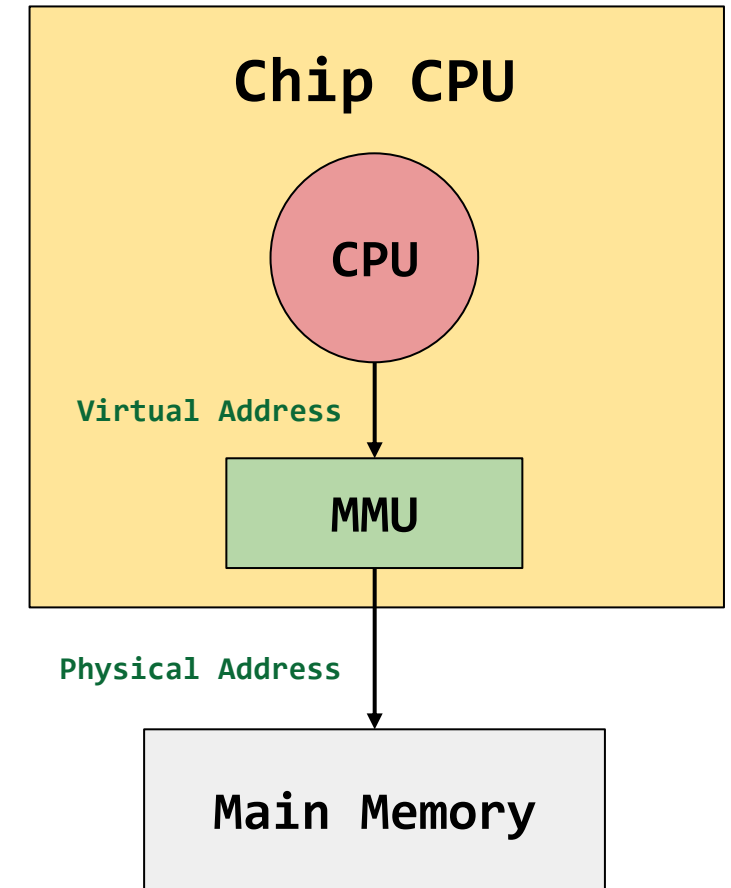
Contextualización

Conceptos claves

- La Virtualización de memoria **requiere soporte en hardware**:
 - Similar al mecanismo de **Ejecución Directa Limitada (Limited Direct Execution)**.
 - **Objetivos**:
 - **Eficiencia**: Uso del recurso lo mejor posible.
 - **Control**: Protección y acceso correcto a memoria.

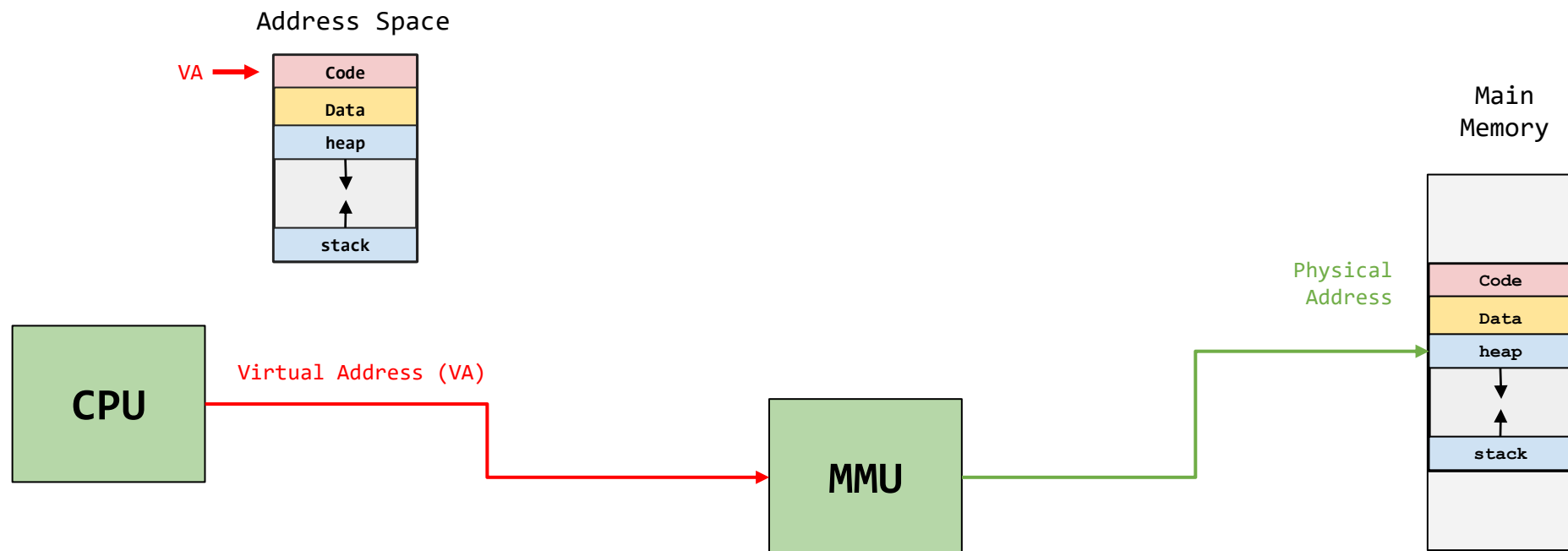
Soporte en hardware

- Modos de CPU
- Interrupciones
- Registros
- Segment tables
- TLB
- Page Tables
- ...



Traducción de direcciones

- Las direcciones del espacio de direcciones son **virtuales**.
- El hardware traduce las **direcciones virtuales** a **direcciones físicas**:
 - La información es **almacenada** en las direcciones físicas.
- El SO debe configurar el hardware en ciertos momentos clave.
 - El SO administra la memoria e interviene cuando es necesario.



Contextualización

Ejemplo

Para el siguiente fragmento de código asuma que la variable `x` está en la dirección 15360 (15K) y que las direcciones de las instrucciones generadas en lenguaje ensamblador (similar al MIPS) están en las direcciones mostradas a continuación.

Lenguaje C

```
void func() {  
    int x = 3000;  
    x = x + 3;  
}
```

Ensamblador

```
128 lw $S0, 15360  
132 addi $S0, $S0, 3  
135 sw $S0, 15360
```

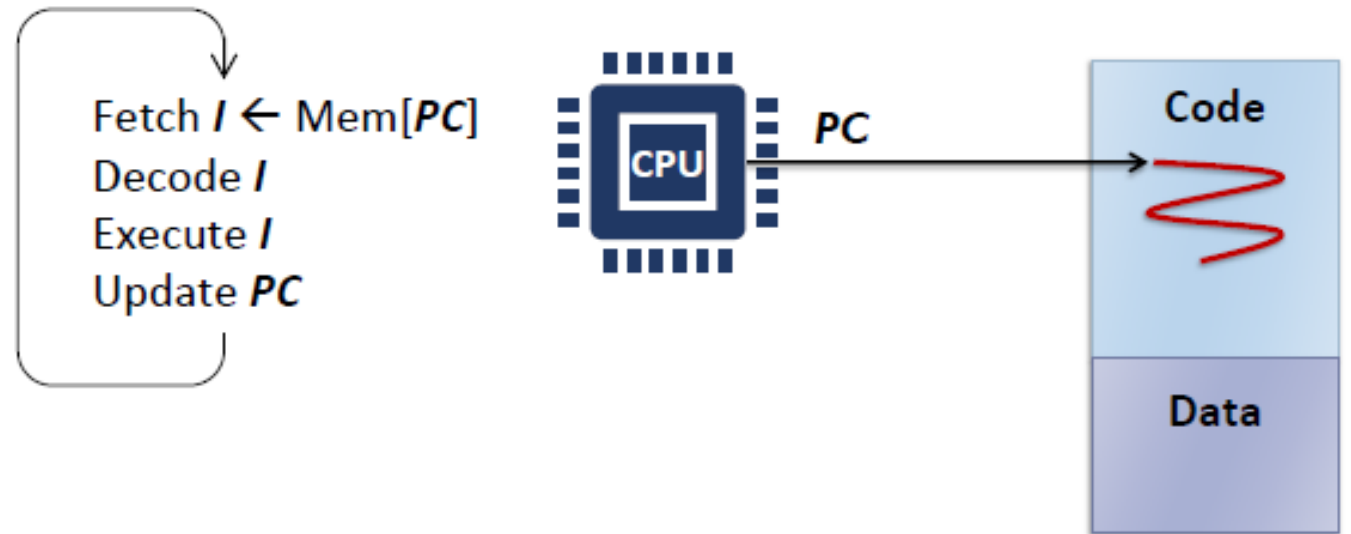
¿Cómo se ejecutaría el código asociado a la instrucción **`x = x + 3;`**?

Contextualización

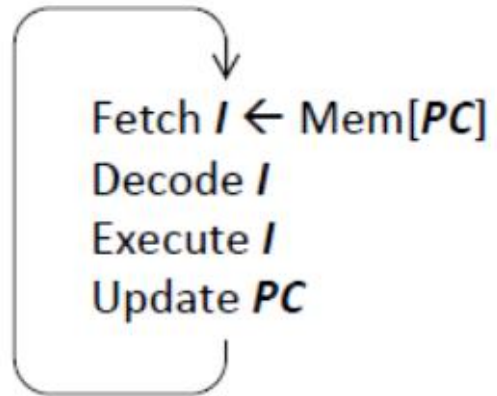
Ejemplo

Ensamblador

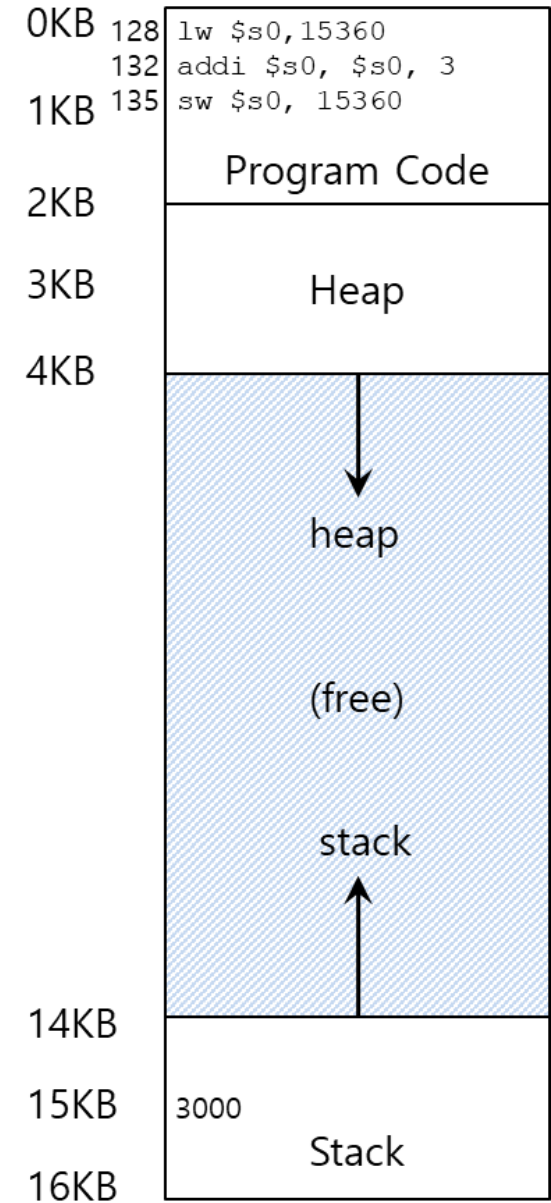
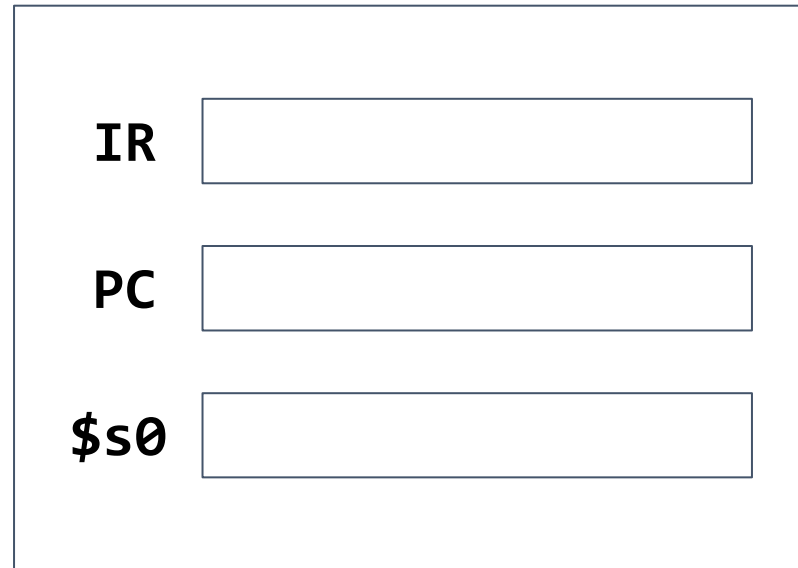
```
128 lw $S0, 15360
132 addi $S0, $S0, 3
135 sw $S0, 15360
```



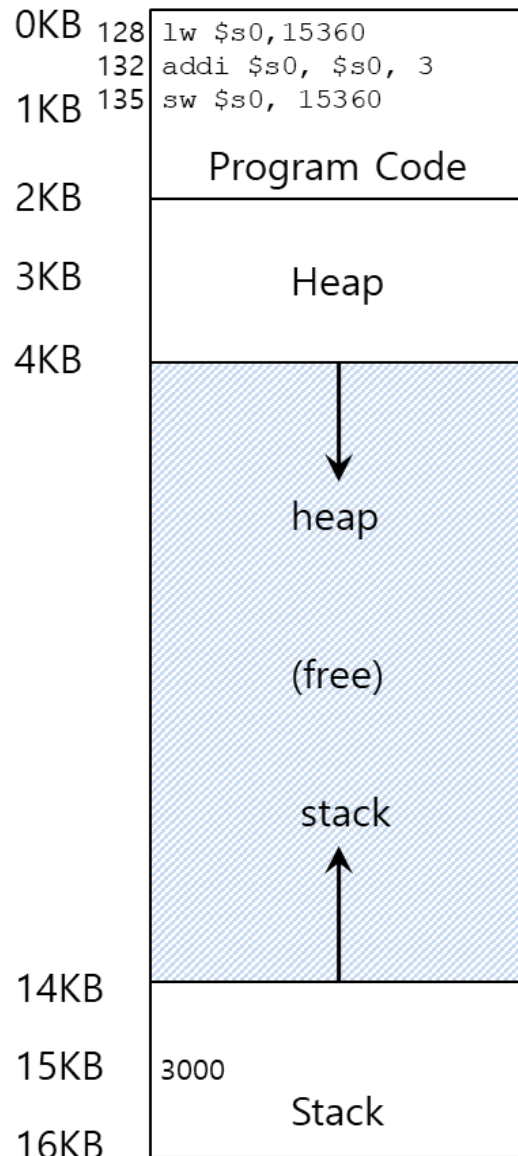
Contextualización



CPU



Contextualización



```
128 lw $S0, 15360
132 addi $S0, $S0, 3
135 sw $S0, 15360
```

- **Trae (fetch)** instrucción desde **dir. 128**
- **Ejecuta** esta instrucción [**Acceso a Mem**]
 - **Carga (load)** dato desde **dir. 15K**

```
128 lw $S0, 15360
132 addi $S0, $S0, 3
135 sw $S0, 15360
```

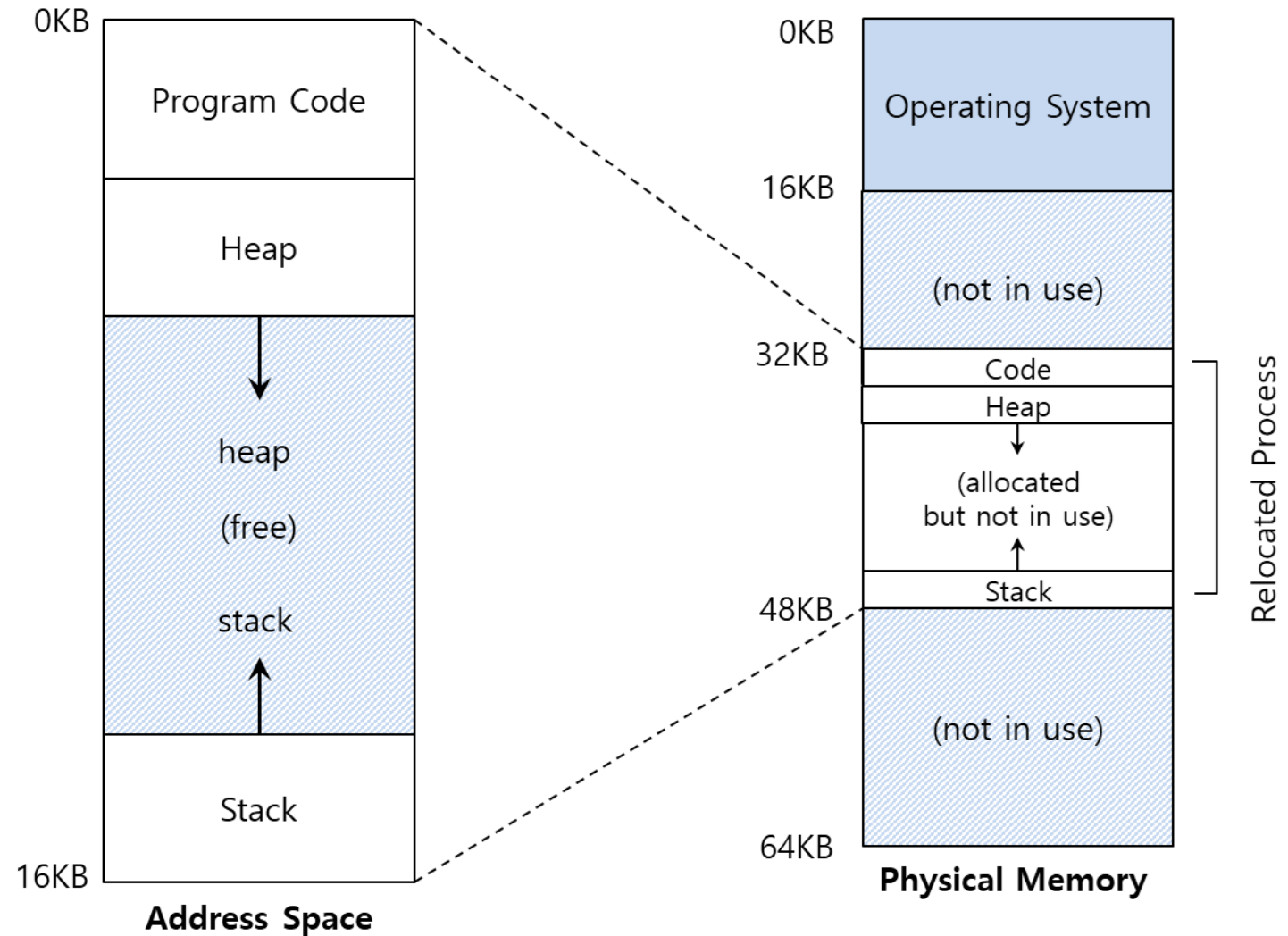
- **Trae (fetch)** instrucción desde **dir. 132**
- **Ejecuta** esta instrucción [**No acceso a Mem**]
 - **Incremento** de **\$s0 en 3**

```
128 lw $S0, 15360
132 addi $S0, $S0, 3
135 sw $S0, 15360
```

- **Trae (fetch)** instrucción desde **dir. 135**
- **Ejecuta** esta instrucción [**Acceso a Mem**]
 - **Guarda (store)** dato desde **dir. 15K**

Reasignación del espacio de direccionamiento

- El SO ubica el proceso en una dirección diferente de 0.
 - Potencialmente **cualquier lugar!**



Contextualización – Pregunta clave

¿Cómo virtualizar memoria de manera eficiente y flexible?



Virtual Address
VA

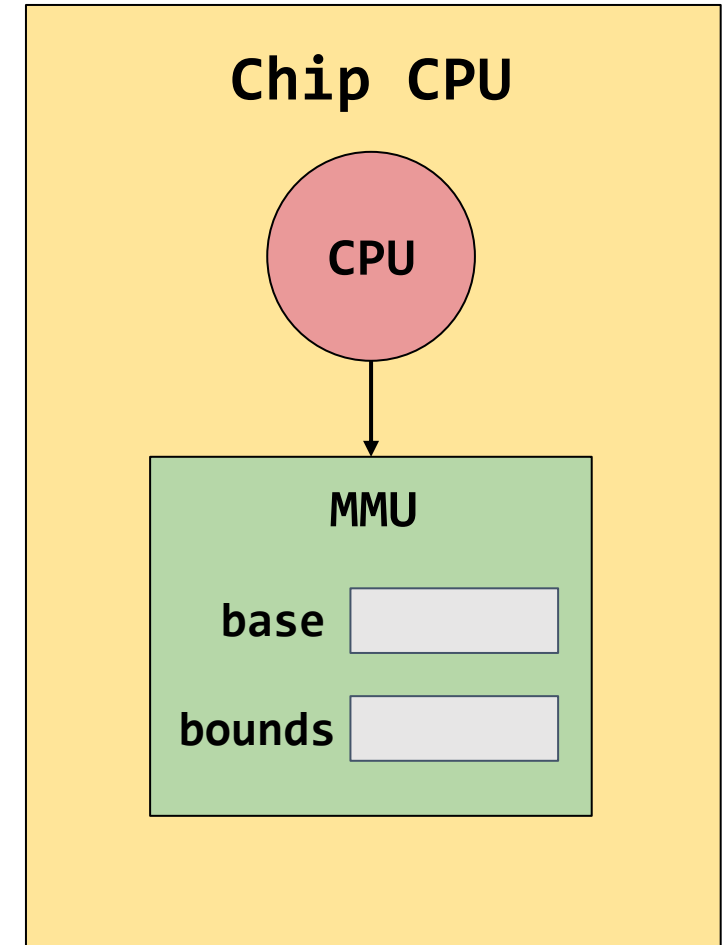
**Address
Translation**

Physical Address
PA

Dynamic Relocation - Reasignación dinámica

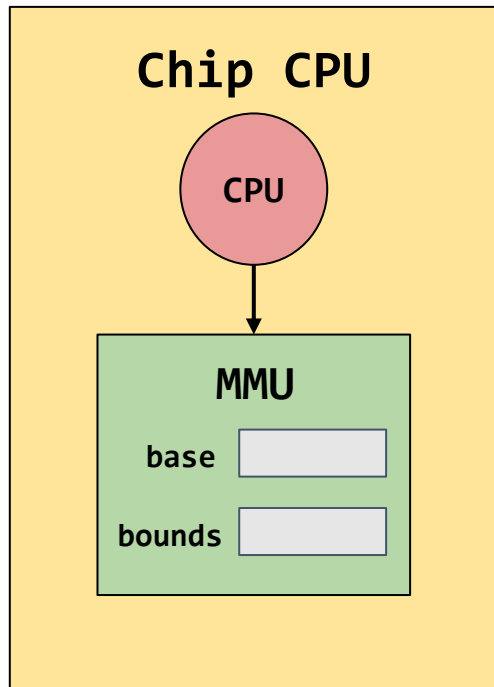
Conceptos

- **Objetivo:** Traducir una dirección virtual a una física al agregar un **offset** fijo cada vez
- También conocida **Hardware-based reallocation** ó **base & bound reallocation**
- Esta técnica hace uso de los registros **base** y **bounds**:
 - **base:** Almacena la dirección física más pequeña.
 - **bounds:** Almacena el tamaño del segmento de memoria virtual asociada al proceso.

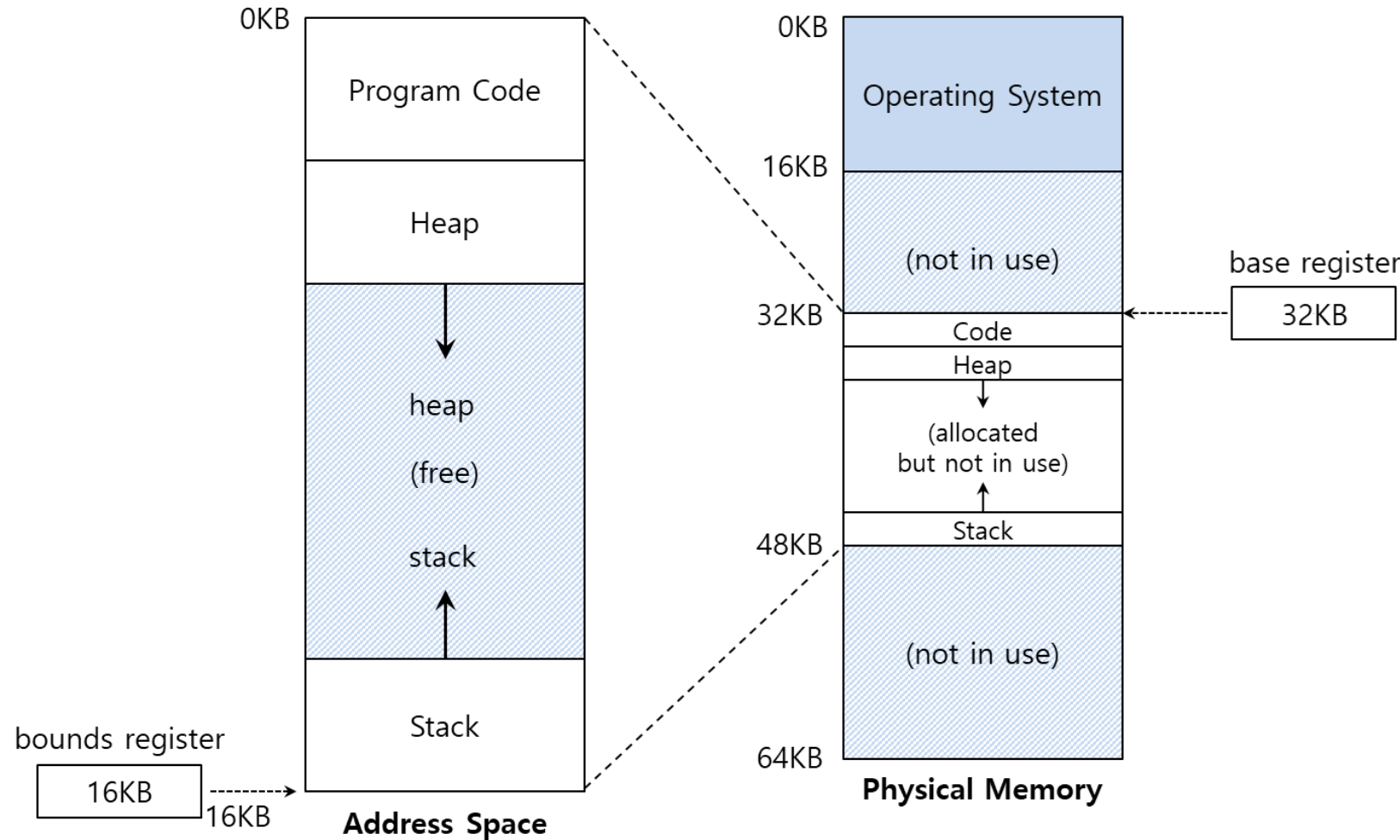


Dynamic Relocation - Reasignación dinámica

MMU (Memory Management Unit)



MMU: Hardware encargado de realizar la traducción de direcciones lógicas a físicas.



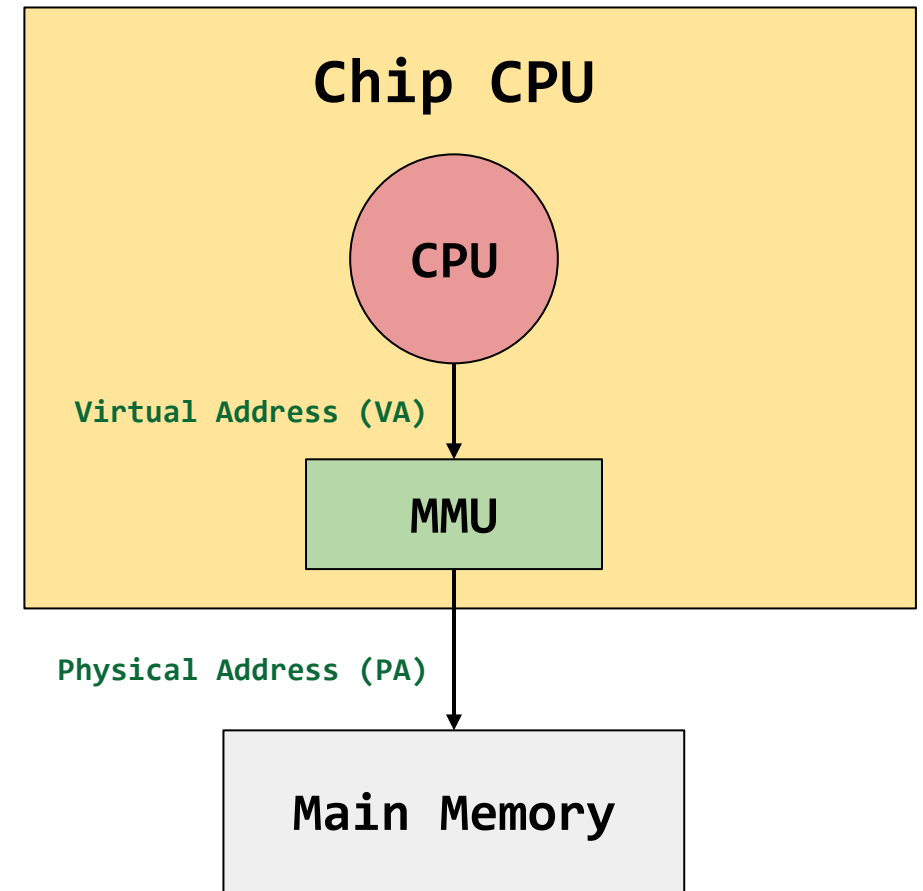
Traducción de direcciones

- Cuando el proceso es creado, el SO decide **dónde** (en memoria física) será **cargada** su imagen de memoria.
 - Fija un valor para el registro base

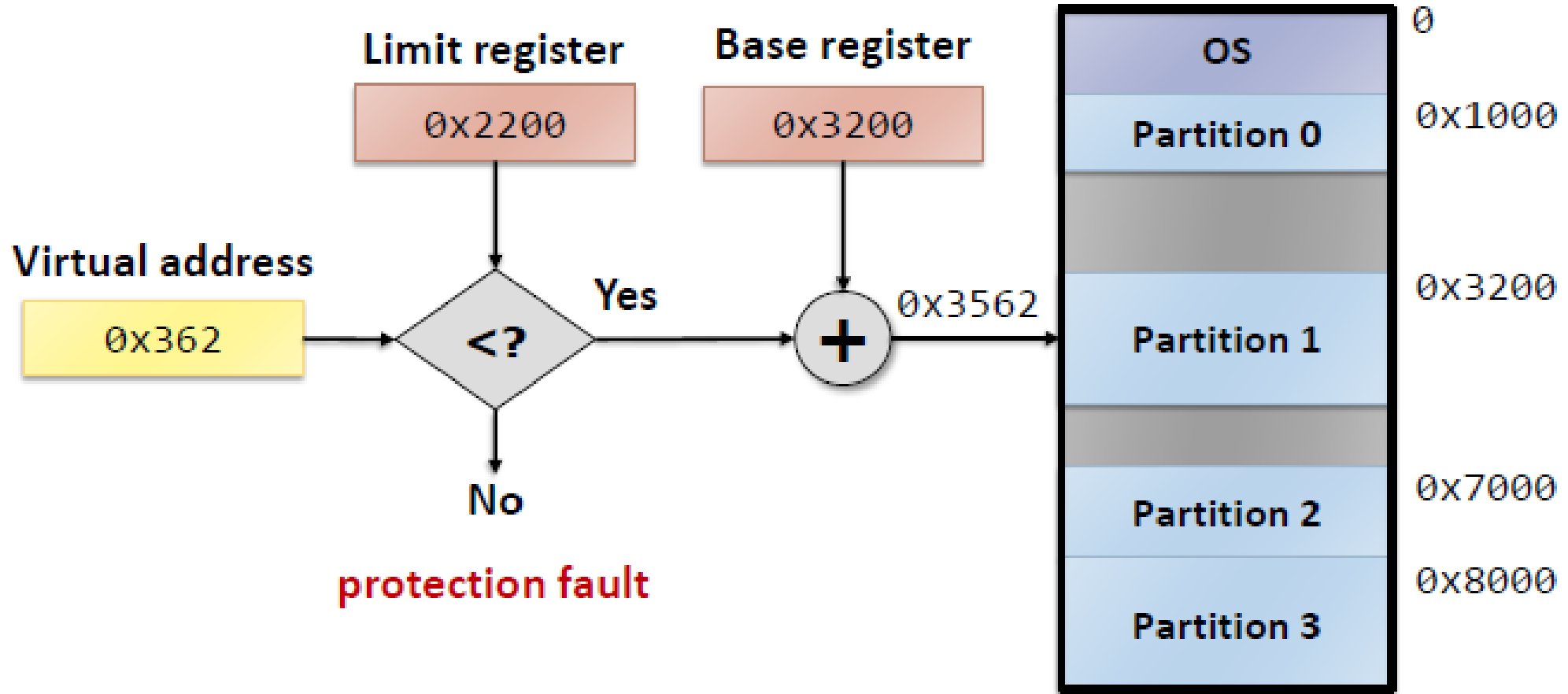
$$PA = VA + base$$

- Las direcciones virtuales no deben ser mayores al valor en el registro bounds (limit) ni negativas.

$$0 \leq VA < bounds$$



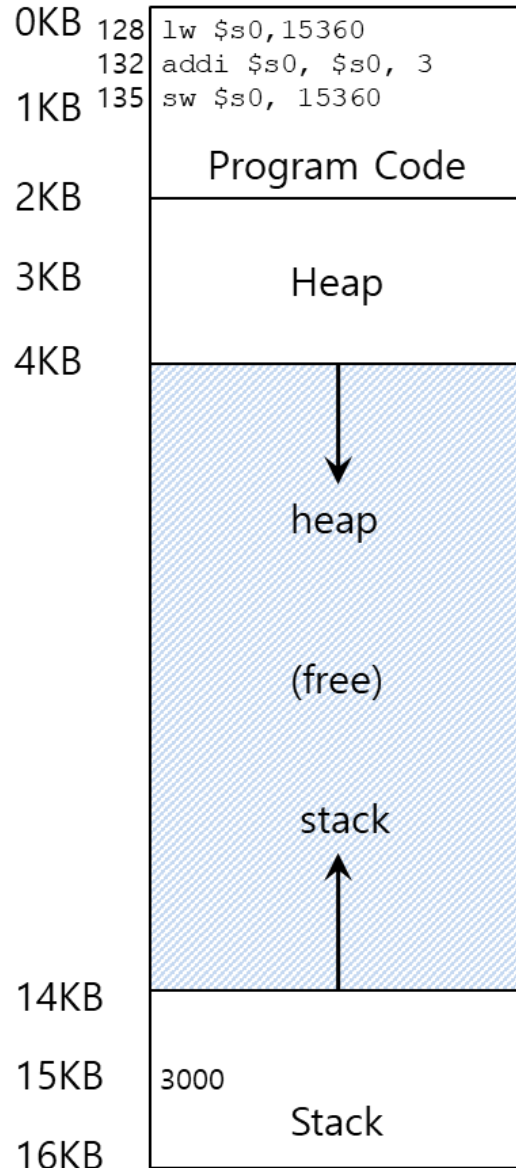
Traducción de direcciones - Formulas



$$0 \leq VA < bounds$$

$$PA = VA + base$$

Traducción de direcciones



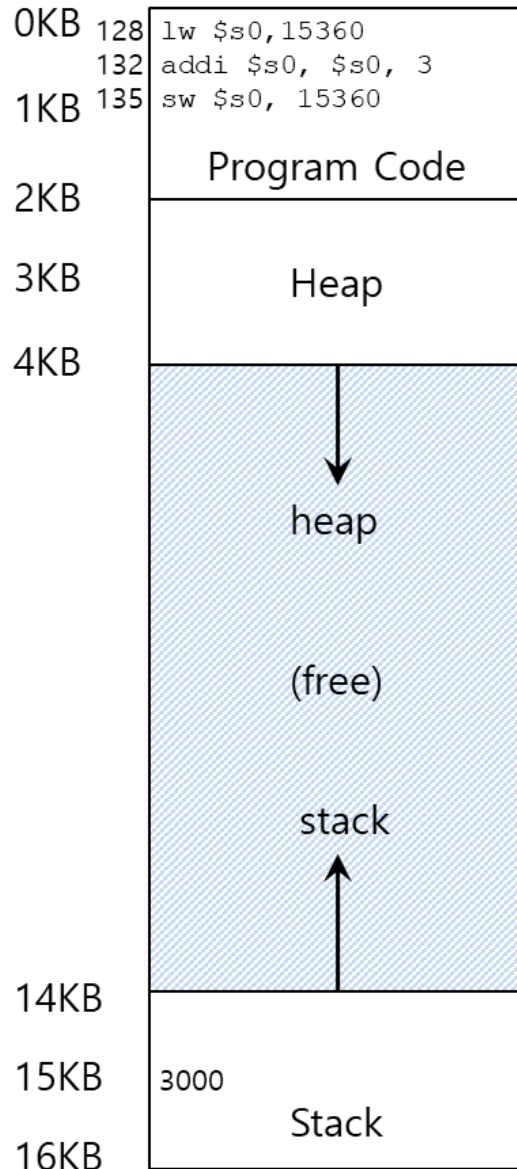
Ejemplo - ¿Como un programa accede a memoria?

Dado el siguiente espacio de memoria asociado a un proceso. Analice qué sucede cuando se ejecuta la instrucción.

128 `lw $S0, 15360`

Asuma que el registro base tiene un valor de 32 K.

Traducción de direcciones



Ejemplo - ¿Como un programa accede a memoria?

128 `lw $s0, 15360`

Para ejecutar la instrucción, se ejecutan dos accesos a memoria.

1. Traer la instrucción de la dirección 128

`I = Fetch[128]`

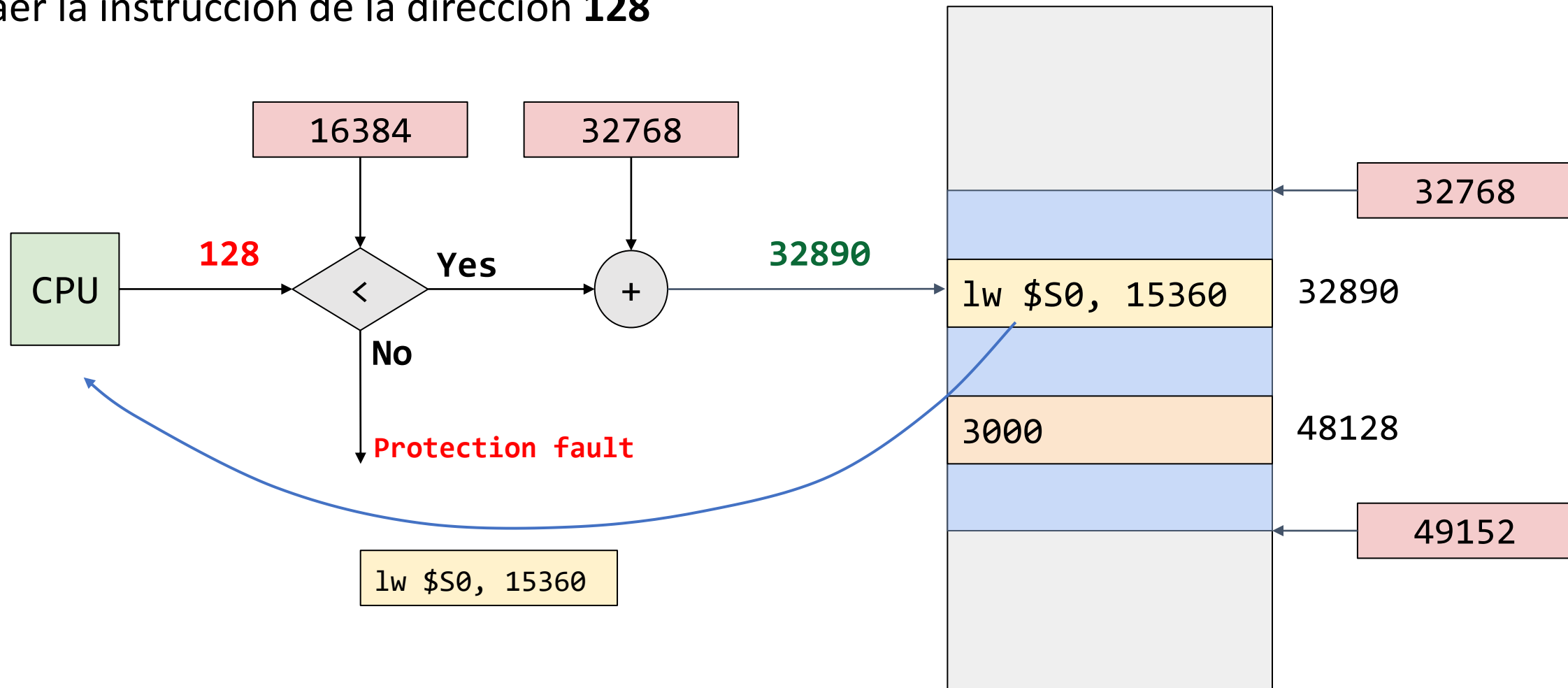
2. Ejecutar la instrucción I: `lw $s0, 15360`

`execute(I)`

Traducción de direcciones

Ejemplo - ¿Como un programa accede a memoria?

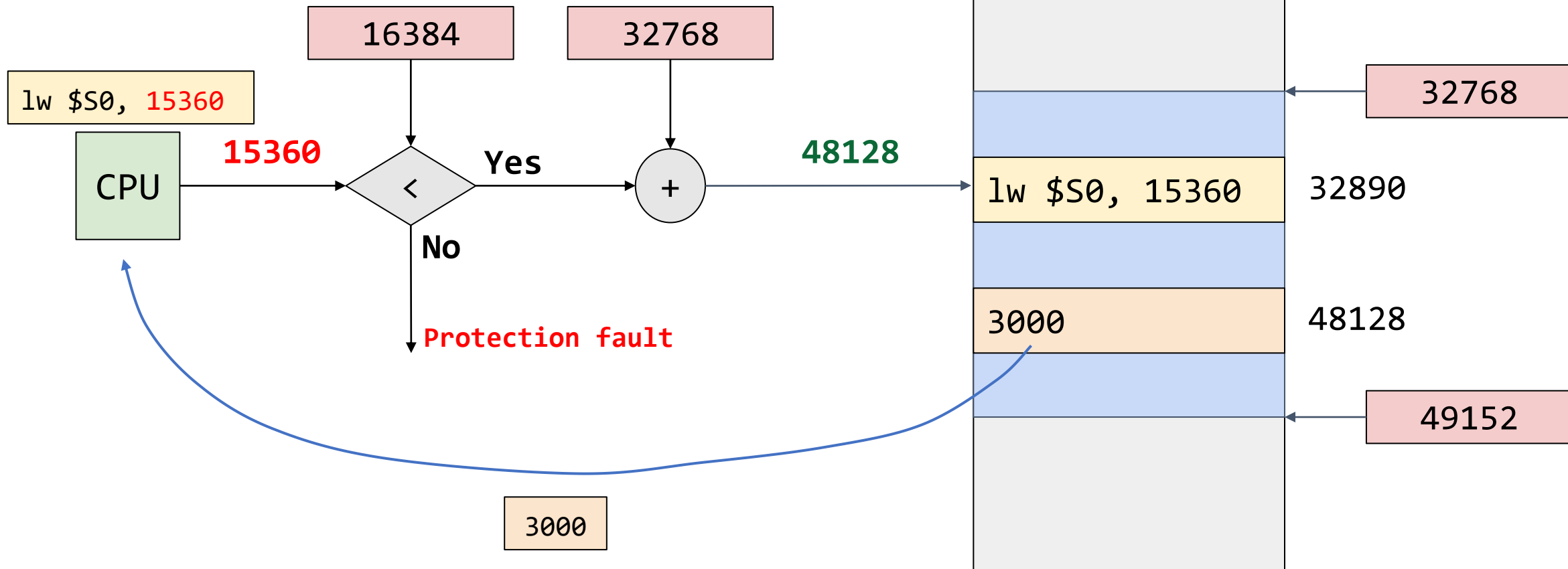
1. Traer la instrucción de la dirección **128**



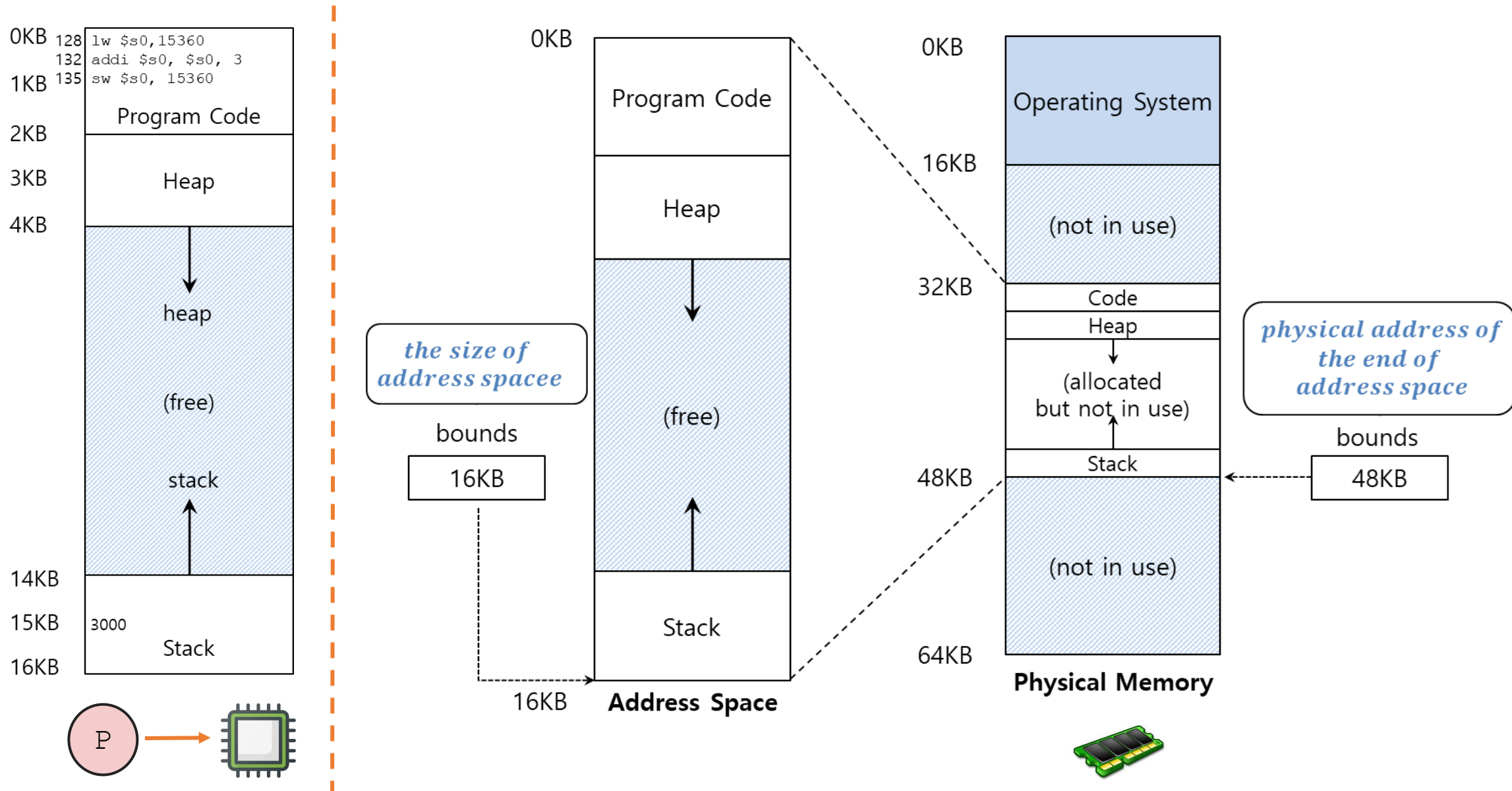
Traducción de direcciones

Ejemplo - ¿Como un programa accede a memoria?

2. Ejecutar la instrucción **l: lw \$s0, 15360**



Traducción de direcciones



Traducción de direcciones

Ejemplo:

Un proceso con un espacio de direccionamiento de 4KB fue asignado a una memoria física a partir de 16KB. Teniendo en cuenta lo anterior, realizar la traducción de direcciones virtuales a físicas mostradas en la siguiente tabla:

Virtual Address (VA)	Physical Address (PA)
0	
1 KB	
3300	
4400	

Traducción de direcciones

Solución:

Virtual Address (VA)	Physical Address (PA)
0	
1 KB	
3300	
4400	

Traducción de direcciones

Ejemplo:

Un proceso con un espacio de direccionamiento de 4KB fue asignado a una memoria física a partir de 16KB. Teniendo en cuenta lo anterior, realizar la traducción de direcciones virtuales a físicas mostradas en la siguiente tabla:

Virtual Address (VA)	Physical Address (PA)
0	16 KB
1 KB	17 KB
3300	19684
4400	Fault (Out of bounds)

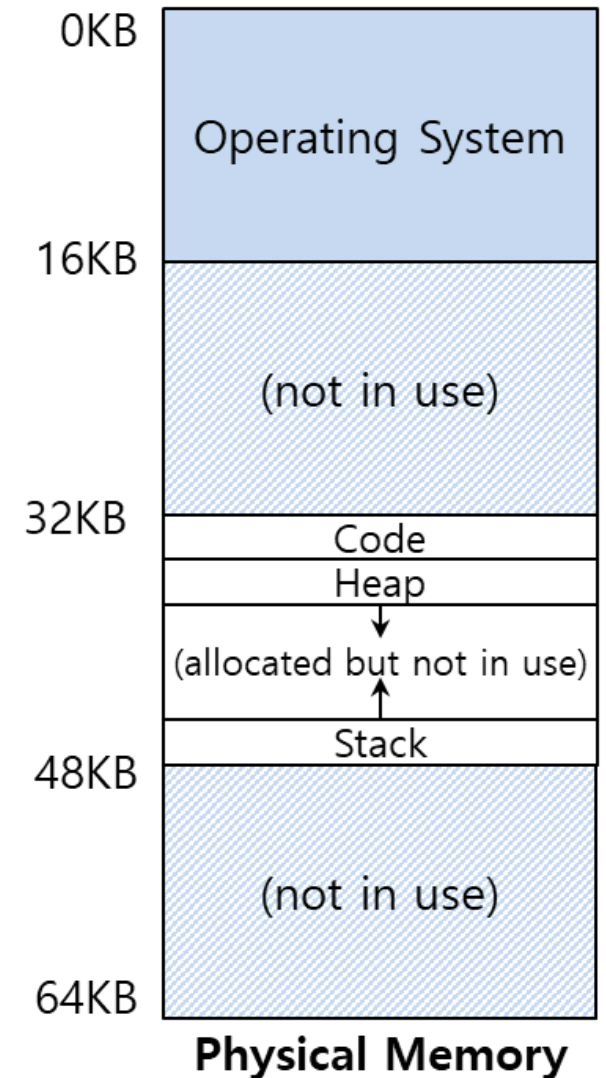
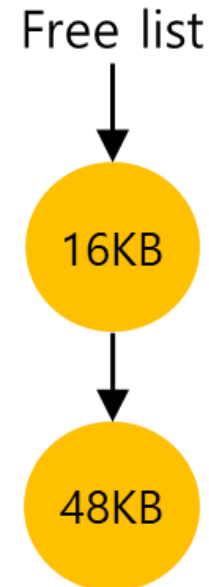
Acciones del sistema operativo

El SO debe realizar acciones para el correcto funcionamiento del sistema

base & bound.

1. Cuando un proceso inicia su ejecución
2. Cuando un proceso termina
3. Cuando ocurre un cambio de contexto

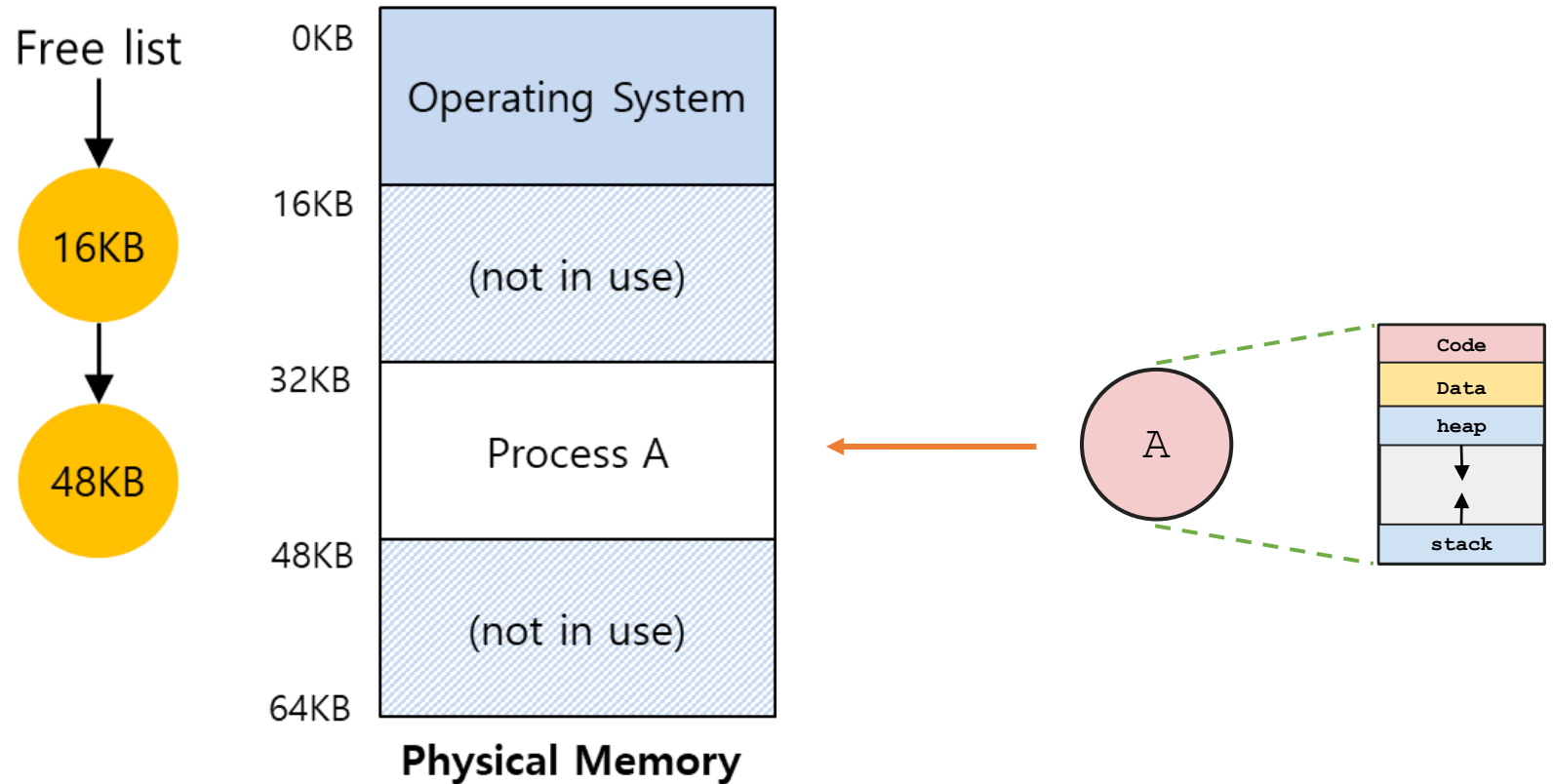
Free list: Lista de rangos de espacio libre en memoria física.



Acciones del sistema operativo

Creación de un proceso

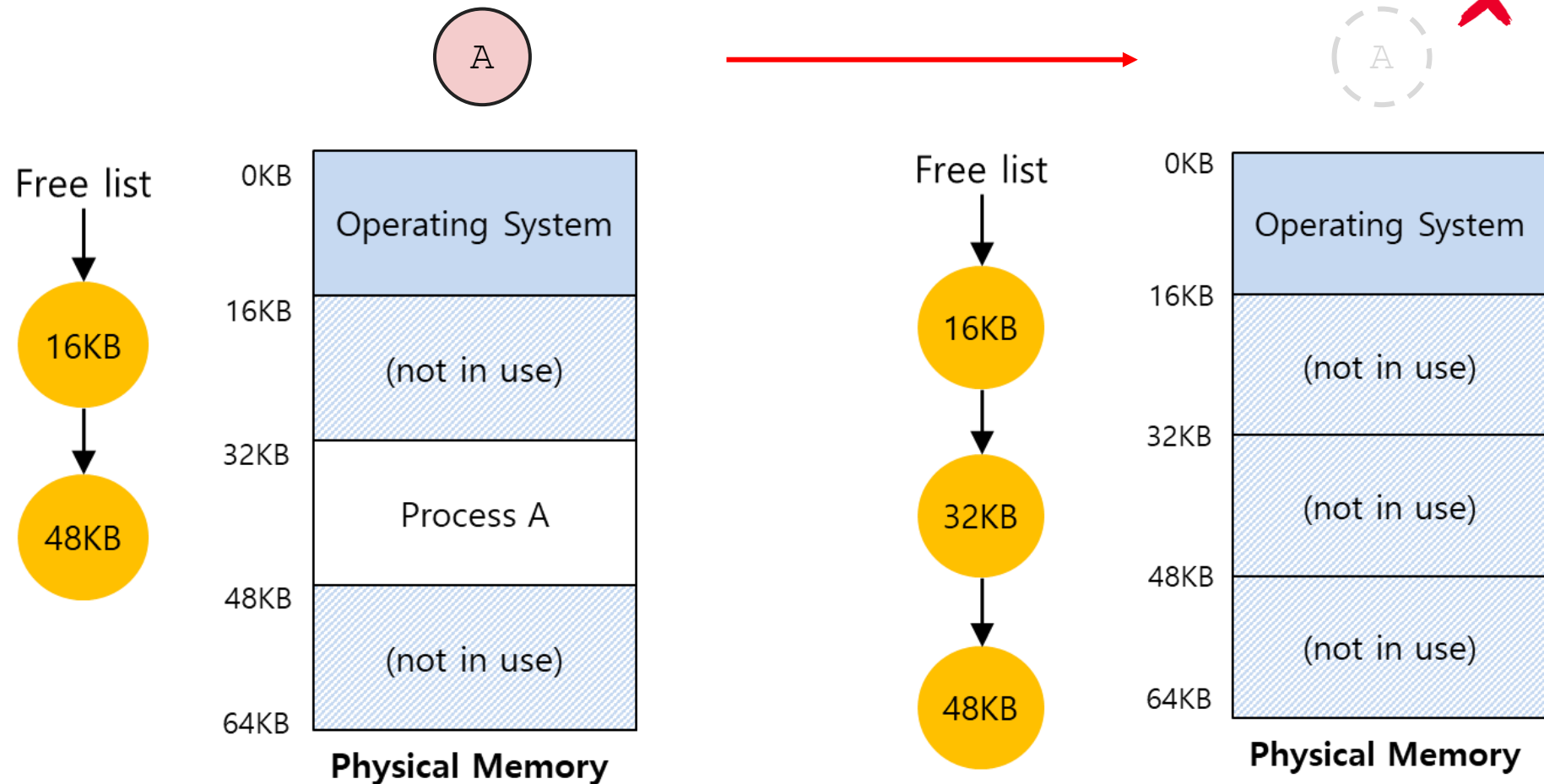
El SO debe encontrar un espacio para la memoria del nuevo proceso (**address space**).



Acciones del sistema operativo

Terminación de un proceso

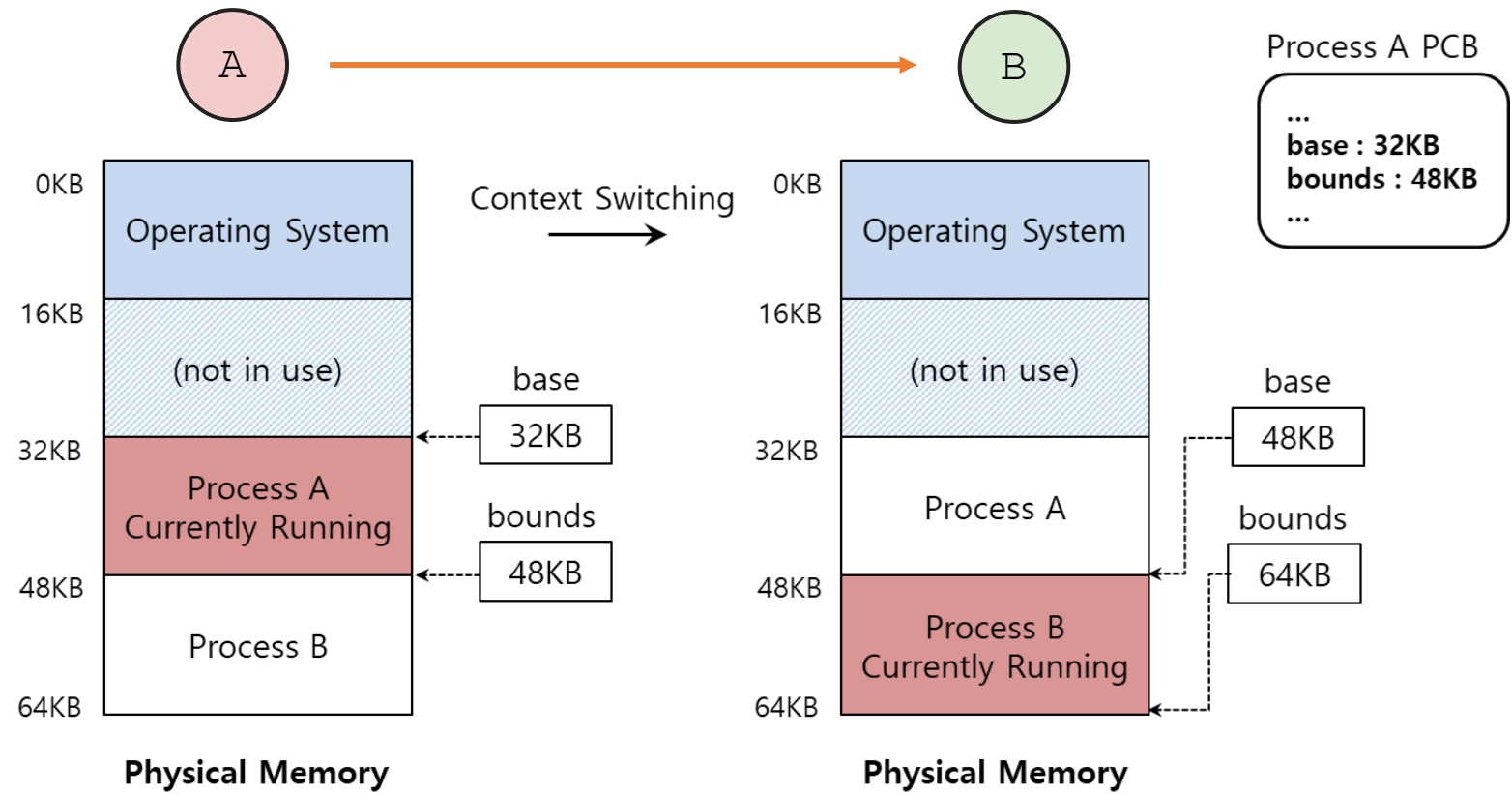
El SO recupera la memoria para su re-uso



Acciones del sistema operativo

Cambio de contexto

El SO almacena y restaura los registros **base** y **bound** en el PCB.



Referencias

- **Operating Systems Three Easy Pieces** ([website](#): Capítulos [1](#) y [2](#))
- Videos de youtube: Clase 1 - Introducción a los sistemas operativos ([link](#)).
- Página de Remzi H. Arpaci-Dusseau ([link](#))
- Operating System Concepts - Tenth Edition ([website](#))
- Operating Systems Notes ([link](#))
- <https://github.com/Aniruddha-Tapas/Operating-Systems-Notes/blob/master/1-Overview.md>
- <https://myaut.github.io/dtrace-stap-book/kernel/proc.html>
- <https://myaut.github.io/dtrace-stap-book/index.html>
- <https://www.technologyuk.net/computing/computer-software/operating-systems/operating-system-utilities.shtml>
- <http://www.it.uu.se/education/course/homepage/os/vt18/module-4/simple-threads/>
- <https://courses.cs.duke.edu/spring19/compsci590.1/slides/>
- <http://csl.snu.ac.kr/courses/4190.307/2020-1/>